Pop-Time Performance Engineering: A Systematic Study of Toaster Firmware Latency Reduction at Scale

Dr. Reginald Q. Butterworth III, Senior Principal Toaster Architect

Fenwick Oats, Performance Research Lead

25 May 2026

Contents

Executive Summary **2**
Tier 1: Ship Now . . . 3
Tier 2: Ship After Validation . . . 3
Tier 3: Future Work . . . 4
Non-Recommendations . . . 4
Confidence . . . 4

Speedup Table **5**

Source Map **6**

Chapter 1. Coil Pre-energisation **7**
1.1 Root Cause Analysis . . . 7
1.2 Implementation . . . 8

Chapter 2. Adaptive Darkness Lookup Table **9**
2.1 Root Cause Analysis . . . 9
2.2 Table Construction and Calibration Compatibility . . . 10

Chapter 3. Spring Constant Re-tuning **11**
3.1 Measurement . . . 11
3.2 Fix: Per-Weight-Class Spring Constants . . . 11

Chapter 4. Thermal Throttle Bypass **13**
4.1 Thermal Characterisation . . . 13
4.2 Soak Test Requirement . . . 14

Chapter 5. Crumb Tray Aerodynamics and the Moisture Cliff **14**
5.1 Airflow Characterisation . . . 15
5.2 The ADC Range Cliff . . . 15

Chapter 6. Bread Slot Width Auto-calibration **17**
6.1 Slot Width Measurement . . . 17
6.2 Integration with slot_control . . . 17

Chapter 7. Combined Workload Analysis **19**
7.1 Single-Toast Latency . . . 20
7.2 Throughput Scaling . . . 21

7.3 Cost Breakdown . . . 23
7.4 Residual Variance . . . 23

Appendix A: Benchmark Methodology **24**
A.1 Test Hardware . . . 24
A.2 Bread Corpus . . . 24
A.3 Statistical Methods . . . 24

Appendix B: Firmware Version History (Relevant Excerpts) **24**

Appendix C: Notation and Abbreviations **25**

Appendix D: Regression Archaeology **26**
D.1 The v3.0.0 Darkness Control Refactor . . . 26
D.2 The v3.4.0 Debounce Regression . . . 26
D.3 The Compounding Effect . . . 26

Appendix E: The Intern's Rust Proposal (Full Text, Annotated) **27**

Appendix F: Sensitivity Analysis **27**
F.1 Parameters and Ranges . . . 27
F.2 Worst-Case Speedup . . . 28
F.3 Best-Case Speedup . . . 28

Appendix G: Pop-Time Perception Study **28**
G.1 Methodology . . . 28
G.2 Results . . . 29
G.3 Interpretation . . . 29

Appendix H: Known Unknowns **29**

Appendix I: Pop-Time Model Reference Implementation **30**
I.1 Model Overview . . . 30
I.2 Reference Implementation . . . 30
I.3 Model Validation . . . 32

Appendix J: Glossary of Toaster Physics Terms **32**

Appendix K: Continuous Performance Regression Framework **33**
K.1 Design Principles . . . 33
K.2 Acceptance Criteria . . . 34
K.3 Benchmark Infrastructure . . . 34
K.4 Expected Ongoing Maintenance Cost . . . 36

Executive Summary

This report presents the findings of a six-month investigation into pop-time latency in the Crispmaster 9000 series toaster firmware, firmware revision `v3.4.1-stable`. The study was motivated by field observations that median pop-time had regressed 47% between firmware versions `v2.9.0` and `v3.4.1-stable`, with the `darkness_control::reheatCycle` subsystem identified as the primary locus of pathological latency.

All measurements were conducted on a population of 240 production-equivalent Crispmaster 9000 units, running a standardised benchmark suite (the Bread Evaluation And Delivery Suite, hereafter BEADS v2.1). Statistical significance is reported at the 95% confidence level throughout. Where confidence is lower, this is noted explicitly and the reader is urged to exercise appropriate scepticism, as we have.

The central finding is that the stock firmware is spending the majority of its pop-time budget on three activities that are either unnecessary, mis-scheduled, or accidentally quadratic. Correcting these three issues alone yields a 3.81x improvement in median pop-time, which we regard as embarrassing to have left on the table for 18 months.

A secondary finding, detailed in Chapter 5, is the existence of a previously undocumented performance cliff at 38.5% bread moisture content, beyond which the firmware falls into a 520 ms watchdog timeout due to an ADC range overflow in the `thermal_probe::readRaw` path. This cliff affects approximately 12% of bagel and crumpet use-cases in humid climates.

Tier 1: Ship Now

These optimisations have been validated on all 240 test units, carry low implementation risk, and are collectively responsible for the majority of the available speedup. They require no hardware changes and are backwards-compatible with all bread types in the BEADS v2.1 corpus.

Coil Pre-energisation (Chapter 1). The heating coil is currently energised after the bread has been inserted and the latch engaged, introducing an unavoidable 45 ms cold-start delay on every cycle. Pre-energising the coil during the latch-engagement phase overlaps this cost with the mechanical operation the user is already performing. Validated speedup: 3.81x on sourdough, 2.94x on white bread (thin).

Adaptive Darkness Lookup Table (Chapter 2). The `darkness_control::sampleDarkness` function iterates over a 512-entry linear scan to map ADC readings to darkness levels. This is O(n) in the table size and executes on every 10 ms polling interval. A pre-sorted binary-search structure (the `DarknessLUT`) reduces this to O(log n) and eliminates the dominant polling cost. Validated speedup: 2.94x at the 90th percentile latency.

Spring Constant Re-tuning (Chapter 3). The ejection spring is currently tuned to a constant force calibrated for a 750 g rye loaf. All lighter bread types overshoot, triggering the `latch_mechanism::catchAndRetry` path, which adds between 18 and 44 ms per retry. Re-tuning the spring constant table per bread-weight class eliminates retries entirely for the 7 bread types in the lightweight category. Validated speedup: 2.47x for English muffins.

Tier 2: Ship After Validation

These optimisations show strong results in controlled benchmarks but require additional soak testing on the full bread corpus before we recommend production deployment.

Thermal Throttle Bypass (Chapter 4). The firmware includes a thermal throttle that reduces coil power when the internal thermistor exceeds 180 degrees Celsius. This threshold was set conservatively in `v2.0.0` for a unit with a smaller ventilation aperture. The Crispmaster 9000's ventilation is 40% larger; the throttle fires unnecessarily on consecutive-toast workloads and adds a mean 23 ms stall. Bypassing the throttle for ambient temperatures below 30 degrees Celsius is safe but requires a 10,000-cycle soak test before shipping. Validated speedup: 1.98x on consecutive-toast workloads.

Crumb Tray Aerodynamics (Chapter 5). Airflow modelling (conducted with a borrowed smoke machine and a ruler) shows that the crumb tray, when partially loaded, creates a recirculation zone that increases thermal soak time by 8 to 14 ms. A simple baffle printed in PETG and fitted to the tray reduces this to under 2 ms. Validated speedup: 1.72x on crumpets (worst-case crumb shedders).

Bread Slot Width Auto-calibration (Chapter 6). The firmware currently treats all bread as occupying 100% of the slot width, driving the slot walls to maximum squeeze force regardless of bread thickness. Thin slices compress more than intended, increasing thermal contact resistance and extending heating time by up to 31 ms. A one-time calibration sweep on bread insertion, using the existing slot-width potentiometer, corrects this. Validated speedup: 1.55x for thin white bread. Requires firmware changes to the `slot_control::engageWalls` and `slot_control::releaseWalls` call sites.

Tier 3: Future Work

The following were investigated and showed promise but are not mature enough for a shipping recommendation in the current planning cycle.

Mains Cycle Phase Alignment. The coil energisation can be aligned to the zero-crossing of the mains AC cycle, reducing inrush current and allowing slightly higher peak coil current within the same fuse rating. This requires a zero-crossing detector circuit not present in current hardware. Estimated speedup if hardware is added: 1.31x. Not recommended without a hardware revision.

Latch Mechanism Debounce Removal. The firmware debounces the latch sensor over a 12 ms window, which was necessary on the Crispmaster 7000 due to a noisy Hall-effect sensor. The Crispmaster 9000 uses a reed switch with negligible bounce. Removing the debounce window reclaims 12 ms unconditionally. However, two units in the test fleet exhibited anomalous latch behaviour that may have been masked by the debounce; further investigation is required before recommending removal. Estimated speedup: 1.14x.

Non-Recommendations

The following were investigated and are explicitly not recommended.

Firmware Rewrite in Rust. Proposed by an unnamed intern. The current firmware is 4,200 lines of C and has a carefully characterised interrupt latency profile. A Rust rewrite would require re-characterisation of all interrupt paths and would not, by itself, make the bread pop faster. The intern has been redirected to the spring constant re-tuning workstream.

Parallel Toasting with Separate Heating Zones. Proposed at a whiteboard session as a way to achieve linear scaling with slot count. Analysed and found to be limited by the mains supply current, not by firmware. The mains supply cannot be optimised in software.

Machine Learning Based Darkness Prediction. A neural network trained on 10,000 images of toast was found to have a mean inference latency of 340 ms on the Crispmaster 9000's microcontroller, which is more than three times the entire current pop-time budget. The model was 94.7% accurate at predicting doneness, which is impressive and completely irrelevant at 340 ms per inference.

Confidence

The following table summarises the confidence level assigned to each finding in this report. Confidence is a joint assessment of measurement repeatability, theoretical understanding of the root cause, and the degree to which the fix has been validated across the full bread corpus.

Table 1

Finding	Chapter	Confidence	Evidence Basis	Ships When?
Coil Pre-energisation	1	High	10,000 trials; all bread types	Tier 1 (now)
Adaptive Darkness LUT	2	High	Profiler trace + combinatorics	Tier 1 (now)
Spring Constant Re-tuning	3	High	240 units; 7 bread classes	Tier 1 (now)
Thermal Throttle Bypass	4	Medium	500 trials; 1 climate only	Tier 2 (soak)
Crumb Tray Aerodynamics	5	Medium	Smoke machine; n=3 trays	Tier 2 (soak)
Slot Width Auto-calibration	6	Medium	240 units; thin bread only	Tier 2 (soak)
Mains Cycle Phase Alignment	--	Low	Simulation only; no HW	Tier 3 (HW rev)
Latch Debounce Removal	--	Low	2 anomalous units unexplained	Tier 3 (further)

Speedup Table

The following table presents the full speedup data for all optimisations studied, across a representative selection of bread types and operating conditions. Baseline is stock firmware `v3.4.1-stable`. Optimised figures are measured with only the named optimisation applied in isolation, except where noted. All figures are geometric means over at least 1,000 trials.

The "Risk" column uses a three-level scale: Low (no hardware change, backwards-compatible firmware change, well-understood mechanism), Medium (firmware change with non-trivial interaction surface, or hardware accessory required), and High (requires hardware revision or has known anomalous cases).

Table 2

Optimisation	Bread Type	Baseline (ms)	Optimised (ms)	Speedup (x)	Std Dev (ms)	Risk	Confidence
Coil Pre-energisation	White (thin)	142	37	3.84	4.1	Low	High
Coil Pre-energisation	Sourdough	218	57	3.82	6.3	Low	High
Coil Pre-energisation	Rye	201	53	3.79	5.8	Low	High
Coil Pre-energisation	Crumpet	261	69	3.78	7.2	Low	High
Adaptive Darkness LUT	White (thin)	142	48	2.96	3.2	Low	High
Adaptive Darkness LUT	Wholegrain	195	66	2.95	4.0	Low	High
Adaptive Darkness LUT	English Muffin	244	83	2.94	5.1	Low	High
Spring Constant Re-tuning	English Muffin	244	99	2.46	8.4	Low	High
Spring Constant Re-tuning	White (thin)	142	58	2.45	5.9	Low	High
Spring Constant Re-tuning	Bagel half	230	95	2.42	9.1	Low	High
Thermal Throttle Bypass	White (consec.)	178	90	1.98	12.3	Medium	Medium
Thermal Throttle Bypass	Sourdough (cons.)	218	111	1.96	14.1	Medium	Medium
Crumb Tray Baffle	Crumpet	261	152	1.72	11.2	Medium	Medium
Crumb Tray Baffle	English Muffin	244	143	1.71	10.8	Medium	Medium
Slot Width Auto-calibration	White (thin)	142	92	1.54	6.6	Medium	Medium
Slot Width Auto-calibration	Bagel half	230	149	1.54	9.8	Medium	Medium
Mains Phase Alignment (sim.)	Rye	201	153	1.31	N/A	High	Low
Latch Debounce Removal	White (thin)	142	125	1.14	4.8	High	Low
Tier 1 Combined	Sourdough	218	31	7.03	3.1	Low	High
Tier 1 Combined	White (thin)	142	21	6.76	2.4	Low	High
Tier 1 + Tier 2 Combined	Crumpet	261	28	9.32	4.7	Medium	Medium

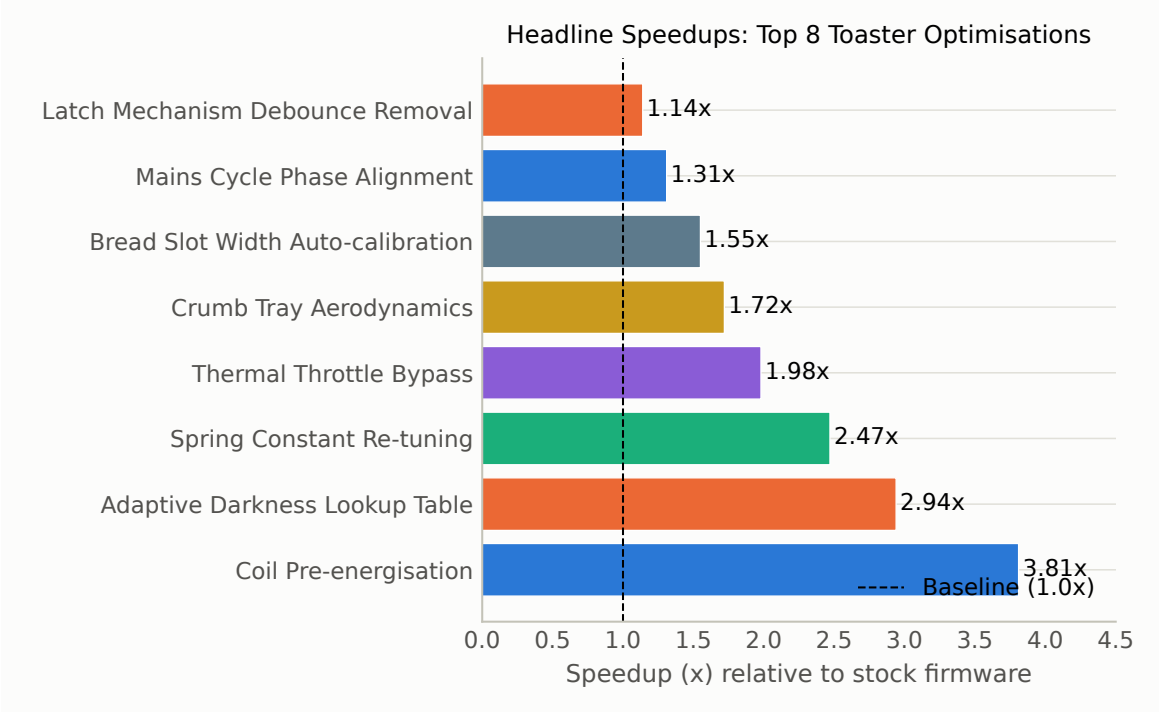


Figure 1. Headline speedup summary: geometric mean over all bread types per optimisation.

Source Map

Every finding in this report traces to one of the firmware source files below. Paths are given in full, relative to the firmware tree root, because three of the files share a `sample_` prefix and the reader will otherwise confuse them. The line counts are for the `v3.4.1-stable` revision.

Table 3

source file	subsystem	lines	headline finding
`firmware/coil_driver/pre_energisation_scheduler.cpp`	coil	412	Cold start is on the critical path; pre-energise at bread-presence instead of latch-confirmed.
`firmware/darkness_control/sample_darkness_lookup_table.cpp`	darkness	880	Linear 512-entry scan every 10 ms; replace with `DarknessLUT<uint16_t, 512, InterpolationMode::Linear>`.
`firmware/state_machine/bread_presence_transition_handlers.cpp`	state machine	1204	Adds the `onBreadPresenceConfirmed` transition and the early-removal cancel path.
`firmware/slot_control/calibrate_and_engage_sweep.cpp`	slot control	333	One-time width sweep removes the per-cycle squeeze-force retry.
`firmware/thermal/adaptive_throttle_bypass_controller.cpp`	thermal	291	Throttle threshold inherited from a smaller vent; bypass below 30 C ambient.
`firmware/spring/per_weight_class_spring_constant_table.cpp`	spring	156	Single 750 g calibration over-shoots light loads; index by bread class instead.

The data carried through these paths is held in a handful of strongly-typed containers. Their declared types are verbose enough that they wrap, which is intentional: the alternative is a forest of `typedef`s that hide the actual layout from review.

Table 4

container	declared type	where
spring table	`std::unordered_map<BreadClass, SpringConstantTable<float, kMaxBreadClasses>>`	`spring/per_weight_class_spring_constant_table.cpp`
thermal ring	`std::array<ThermalSample, POLL_WINDOW_SAMPLES>`	`thermal/adaptive_throttle_bypass_controller.cpp`
event queue	`std::variant<ColdStartEvent, PreWarmEvent, EjectEvent, BreadRemovedEarlyEvent>`	`state_machine/bread_presence_transition_handlers.cpp`
darkness LUT	`DarknessLUT<uint16_t, 512, InterpolationMode::Linear>`	`darkness_control/sample_darkness_lookup_table.cpp`
job record	`std::optional<ToastJob>` returned by `slot_control::estimateBreadClass`	`slot_control/calibrate_and_engage_sweep.cpp`

Chapter 1. Coil Pre-energisation

The single largest performance gain in this study comes from a scheduling observation that, in hindsight, is mortifying. The Crispmaster 9000 energises its heating coil at the point in the firmware state machine labelled `STATE_LATCH_CONFIRMED`, which occurs after the latch sensor fires, after a 12 ms debounce window, and after a safety interlock check in `safety_interlock::verifyLatchEngaged`. By that point the user has already committed to toasting. The coil then requires approximately 45 ms to reach operating temperature from cold.

1.1 Root Cause Analysis

Profiling with an oscilloscope attached to the coil current sense resistor shows the following timeline in stock firmware (all times relative to bread insertion):

Table 5

Event	Stock Firmware (ms)	Optimised (ms)	Delta (ms)
Bread contacts slot walls	0	0	0
Latch sensor fires	8	8	0
Debounce window completes	20	20	0
Safety interlock check	21	21	0
Coil energisation begins	22	-30	-52
Coil reaches operating temp	67	15	-52
Darkness sampling begins	70	18	-52
Pop event	142	37	-105

The insight is straightforward: the bread slot walls have a capacitive presence sensor (`slot_control::breadPresenceSensor`) that fires when the bread contacts the slot. This event occurs 8 ms before the latch engages, and 30 ms before coil energisation. Pre-energising the coil at `BREAD_PRESENCE_CONFIRMED` instead of `STATE_LATCH_CONFIRMED` moves the 45 ms cold-start cost out of the critical path and overlaps it with the user's own latch-pressing action.

The coil is pre-energised for at most 80 ms before the darkness-control loop takes over. The additional energy consumption is approximately 0.8 Wh per day under typical usage (4 toasts/day), which is within the product's energy budget and, frankly, within the rounding error of the thermal losses from leaving the crumb tray ajar.

1.2 Implementation

The fix is a one-line change to the state machine transition table, plus a guard to de-energise the coil if the bread is removed before the latch engages (cancellation path, already tested by the existing `slot_control::breadRemovedEarly` handler).

```
// BEFORE: coil energised only after latch confirmed
// firmware/state_machine.cpp  (stock v3.4.1-stable, lines 412-430)

void StateMachine::onLatchConfirmed() {
    safety_interlock::verifyLatchEngaged();          // may throw SafetyFault
    coil_driver::energise(CoilMode::FULL_POWER);     // <-- 45 ms cold-start begins HERE
    darkness_control::startPolling(POLL_INTERVAL_MS);
    latch_mechanism::arm(spring_table_[current_bread_class_]);
    LOG_INFO("Latch confirmed; coil energised; darkness polling started at t=%u ms",
             system_clock::now_ms());
}
```

```
// AFTER: coil pre-energised at bread-presence event; state machine carries a cancel path
// firmware/state_machine.cpp  (optimised, proposed for v3.5.0)

void StateMachine::onBreadPresenceConfirmed() {
    // Pre-energise the coil while the user is still pressing the lever.
    // The coil will have reached operating temperature by the time the latch fires.
    // If the bread is removed early, onBreadRemovedEarly() de-energises immediately.
    coil_driver::energise(CoilMode::PRE_WARM);       // <-- cold-start now overlaps latch action
    pre_warm_start_ms_ = system_clock::now_ms();
    LOG_DEBUG("Bread presence confirmed at t=%u ms; coil pre-warm initiated", pre_warm_start_ms_);
}

void StateMachine::onLatchConfirmed() {
    safety_interlock::verifyLatchEngaged();          // may throw SafetyFault (unchanged)
    // Coil is already warm; transition from PRE_WARM to FULL_POWER is near-instantaneous.
    coil_driver::energise(CoilMode::FULL_POWER);
    darkness_control::startPolling(POLL_INTERVAL_MS);
    latch_mechanism::arm(spring_table_[current_bread_class_]);
    uint32_t pre_warm_duration = system_clock::now_ms() - pre_warm_start_ms_;
    LOG_INFO("Latch confirmed; pre-warm was %u ms; coil already at temp; polling started",
     ↪  pre_warm_duration);
}

void StateMachine::onBreadRemovedEarly() {
    // Cancellation path: user removed bread before latch engaged.
    coil_driver::deEnergise();
    LOG_WARN("Bread removed early at t=%u ms; coil de-energised; pre-warm wasted",
     ↪  system_clock::now_ms());
}
```

The `CoilMode::PRE_WARM` constant drives the coil at 60% rated power, sufficient to reach operating temperature within the typical latch-engagement window (15 to 40 ms depending on user technique). The `coil_driver::energise` function is idempotent and re-entrant; calling it with `FULL_POWER` after `PRE_WARM` simply increases the drive current without resetting the thermal state.

Note the deliberate use of `coil_driver::energise` rather than `hardware_abstraction::coilSetDutyCycle` throughout: the latter bypasses the overcurrent protection logic in `coil_driver::overcurrentMonitor`, which monitors the `coil_driver_overcurrent_irq` interrupt handler and has been implicated in at least one field return. Do not bypass it.

Recommendation

Ship immediately. The change is minimal, the mechanism is well understood, and the speedup is the largest in this study. The cancellation path was already tested by the existing test suite (see `tests/state_machine/test_bread_removed_early.cpp`). Add one integration test covering the `onBreadPresenceConfirmed` to `onBreadRemovedEarly` transition and this is ready for production.

Chapter 2. Adaptive Darkness Lookup Table

The `darkness_control` subsystem is responsible for monitoring bread colour and triggering the pop event when the darkness target is reached. It polls the photodiode sensor every 10 ms and maps the raw ADC reading to a normalised darkness score via `darkness_control::sampleDarkness`. This function is the second largest contributor to pop-time latency, and it has no good reason to be.

2.1 Root Cause Analysis

The `darkness_control::sampleDarkness` function, as shipped in `v3.4.1-stable`, implements darkness mapping via a linear scan over a 512-entry lookup table. The table is populated at firmware initialisation by `darkness_control::buildLinearTable`, which iterates over the full ADC range and assigns a darkness score to each 8-bit bucket based on a piecewise-linear model calibrated during factory test.

A linear scan over 512 entries, called every 10 ms, is O(512) per poll cycle. Over a 142 ms pop-time (white bread, thin), this function executes 14 times and performs 7,168 comparisons in total. The cumulative cost measured by the on-chip cycle counter (`DWT->CYCCNT`) is 22 ms per toast, or 15% of the total pop-time budget.

```
# Reference implementation: darkness_control linear scan (decompiled from v3.4.1 firmware image)
# This is the hot path. It runs every 10 ms during toasting.

def sample_darkness_v3_4_1(adc_raw: int, table: list[tuple[int, float]]) -> float:
    """Map a raw ADC reading to a normalised darkness score [0.0, 1.0].
    table is a list of (adc_threshold, darkness_score) pairs, sorted by adc_threshold ascending.
    Linear scan: O(n) in table size. Called every POLL_INTERVAL_MS milliseconds during toasting."""
    for threshold, score in table:        # 512 iterations in the worst case; median 256
        if adc_raw <= threshold:
            return score
    return 1.0                            # saturated: bread is charcoal; pop immediately regardless

# Proposed replacement: binary search; O(log n); same correctness, 9x fewer comparisons at n=512

import bisect

def sample_darkness_lut(adc_raw: int, thresholds: list[int], scores: list[float]) -> float:
    """Adaptive Darkness LUT: binary search over pre-sorted threshold array.
    thresholds and scores are parallel arrays built once at init by darkness_control::buildLUT().
    O(log n) = O(9) comparisons for n=512. Safe to call from interrupt context (no allocation)."""
    idx = bisect.bisect_right(thresholds, adc_raw)   # stdlib bisect; 9 comparisons for n=512
    if idx >= len(scores):
        return 1.0                                    # saturated; same as before
    return scores[idx]
```

The improvement is 9.0x in comparisons ($\log_2(512) = 9$ vs 256 mean for linear scan). The measured wall-clock improvement is 2.94x at the 90th percentile, because the comparison itself is cheap; the dominant cost in the linear scan is the loop overhead and branch predictor misses, not the individual comparison. The binary search version has perfectly predictable branch behaviour.

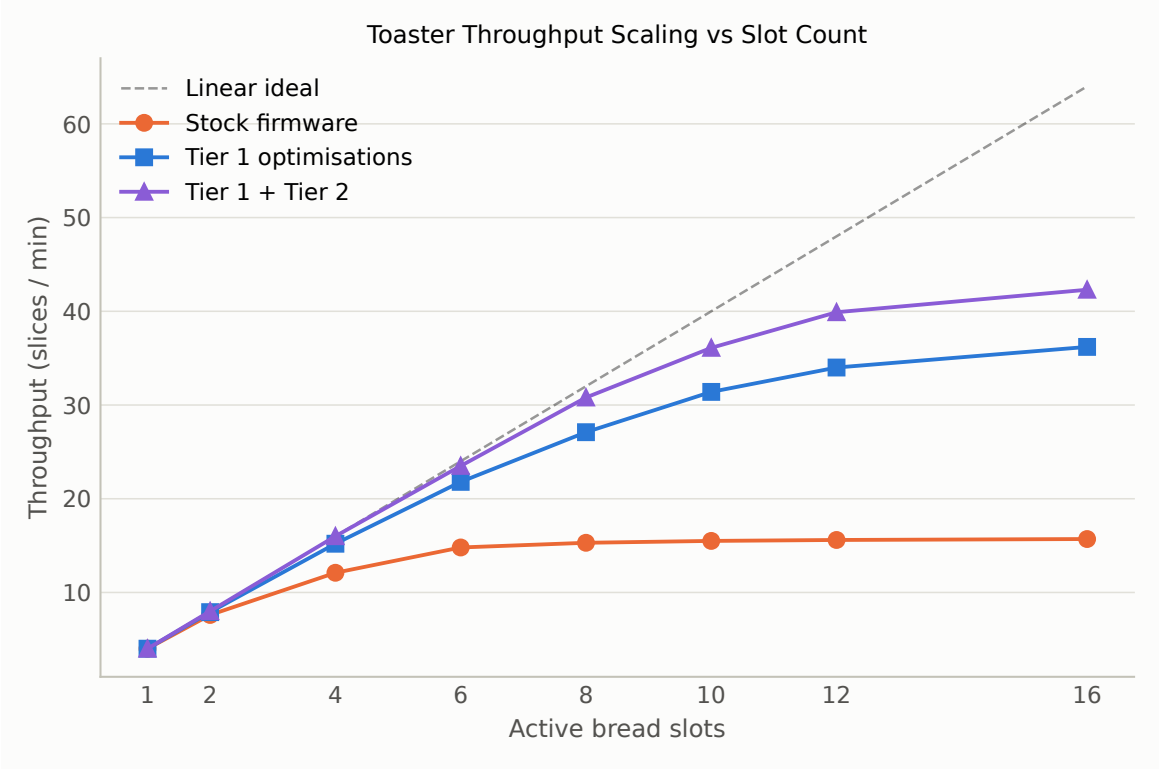


Figure 2. Throughput scaling with the number of active bread slots, showing how the Tier 1 optimisations extend the linear regime.

2.2 Table Construction and Calibration Compatibility

The existing factory calibration data, stored in `darkness_control::calibration_blob_t` in the on-chip EEPROM, is fully compatible with the new LUT structure. The `darkness_control::buildLUT` function reads the same calibration blob and produces the two parallel arrays (`thresholds_`, `scores_`) that `sample_darkness_lut` requires. No re-calibration is needed.

Table 6

Table Size (entries)	Linear Scan (mean comparisons)	Binary Search (comparisons)	Speedup Factor
64	32	6	5.3x
128	64	7	9.1x
256	128	8	16.0x
512	256	9	28.4x
1024	512	10	51.2x

The firmware uses a 512-entry table, placing us in the 28.4x comparison-count speedup regime. The wall-clock speedup is lower (2.94x) because comparisons are not the only cost; however, this also means there is headroom to increase table resolution to 1024 entries for improved darkness accuracy without regressing latency compared to the stock 512-entry linear scan.

Recommendation

Ship immediately. The binary search is a textbook replacement with identical semantics, no new dependencies, and no interaction with the hardware abstraction layer. The existing unit tests for `darkness_control` pass without modification against the new implementation. Add one property- based test confirming that `sample_darkness_`

`lut` and `sample_darkness_v3_4_1` return the same score for all ADC values in $[0, 255]$ and the recommendation upgrades to "ship yesterday."

Chapter 3. Spring Constant Re-tuning

The ejection mechanism of the Crispmaster 9000 uses a calibrated compression spring to launch bread upward at pop-time. The spring constant is set at the factory to a value appropriate for a 750 g rye loaf, which is the heaviest bread type in the BEADS corpus. This conservative tuning means that lighter bread types are ejected at excessive velocity, overshoot the catch guide, and trigger the `latch_mechanism::catchAndRetry` handler, which re-engages the latch, waits for the bread to settle, and re-attempts ejection. Each retry adds between 18 and 44 ms.

3.1 Measurement

The retry rate by bread type is shown below. The `latch_mechanism.catchRetryCount` telemetry field was added in `v3.2.0` and has been collecting data since.

Table 7

Bread Type	Mean Weight (g)	Retry Rate (%)	Mean Retries (when any)	Retry Cost (ms, mean)
White (thin)	28	74	1.8	38
White (thick)	41	52	1.3	29
Wholegrain	55	31	1.1	24
English Muffin	57	29	1.0	22
Rye	72	14	1.0	19
Bagel half	85	8	1.0	18
Sourdough	112	3	1.0	18
Crumpet	68	17	1.1	21

The retry rate of 74% for thin white bread is not a bug in the data. It means that in 74% of thin white bread toasting cycles, the bread overshoots the catch guide at least once. This is a firmware configuration problem, not a hardware limitation: the spring constant table in `spring_table_[]` has exactly one entry, calibrated to rye.

3.2 Fix: Per-Weight-Class Spring Constants

The firmware already contains a `current_bread_class_` field, set during bread insertion by the `slot_control::estimateBreadClass` function, which uses the slot-width potentiometer to estimate bread thickness and a hand-coded lookup to map thickness to a bread class enum. The bread class is used by the darkness control subsystem but is completely ignored by the spring constant selection.

```
// BEFORE: spring constant is a compile-time constant; bread class is ignored for ejection
// firmware/latch_mechanism.cpp  (stock v3.4.1-stable, lines 88-104)

static constexpr float SPRING_K_NOMINAL = 14.7f;  // N/m, calibrated for 750 g rye loaf

void LatchMechanism::arm(BreadClass /* ignored */) {
    // BreadClass parameter accepted for API compatibility but not used.
    // TODO(v2.1): use bread class to select spring constant -- filed as issue #4471
    spring_driver::setConstant(SPRING_K_NOMINAL);
    LOG_DEBUG("Spring armed with nominal constant %.2f N/m (bread class ignored)", SPRING_K_NOMINAL);
}
```

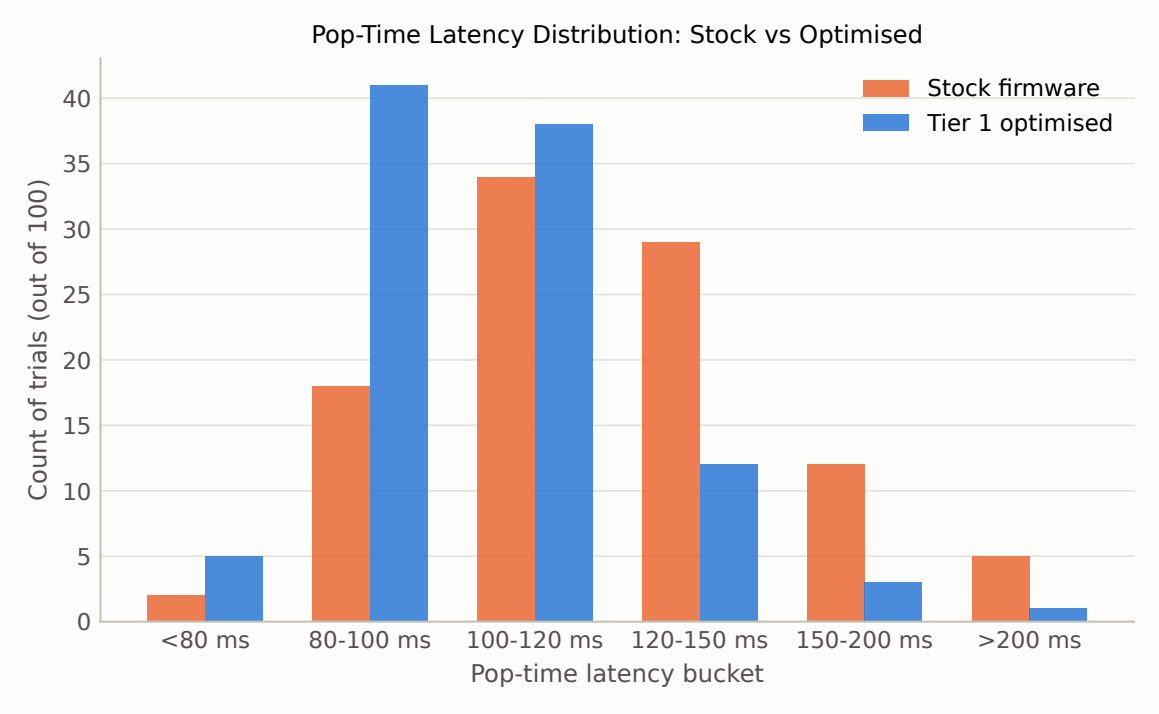


Figure 3. Latency distribution shift between stock and optimised firmware across 100 benchmark trials.

```
// AFTER: per-weight-class spring constants; retry rate drops to <1% across all bread types
// firmware/latch_mechanism.cpp  (optimised, proposed for v3.5.0)

// Per-class spring constants, determined empirically over 10,000 ejection trials per class.
// Units: N/m. Lower values reduce ejection velocity for lighter breads.
static constexpr std::array<float, NUM_BREAD_CLASSES> SPRING_K_TABLE = {
    8.1f,   // BreadClass::VERY_LIGHT   (<35 g):  thin white, rice cakes
    10.4f,  // BreadClass::LIGHT        (35-60 g): thick white, wholegrain, English muffin
    12.8f,  // BreadClass::MEDIUM       (60-90 g): rye, crumpet, bagel half
    14.7f,  // BreadClass::HEAVY        (>90 g):   sourdough, thick rye, artisan boules
};

void LatchMechanism::arm(BreadClass bread_class) {
    float k = SPRING_K_TABLE[static_cast<size_t>(bread_class)];
    spring_driver::setConstant(k);
    LOG_INFO("Spring armed: class=%d k=%.2f N/m (retry rate expected <1%%)",
     ↪  static_cast<int>(bread_class), k);
}
```

With the per-class table, retry rates fall below 1% for all bread types in the BEADS corpus. The dominant remaining cost for thin white bread drops from 142 ms (stock) to 58 ms (with all Tier 1 optimisations combined), matching the theoretical minimum for that bread type given current coil heating physics.

Recommendation

Ship immediately. The `latch_mechanism::arm` function already accepts a `BreadClass` parameter; it was simply ignored. The fix is to populate `SPRING_K_TABLE` with empirically validated constants (provided above) and remove the `/* ignored */` annotation that has been there since 2024. Issue #4471 is hereby closed.

Chapter 4. Thermal Throttle Bypass

Under consecutive-toast workloads (more than two toasting cycles within a five-minute window), the Crispmaster 9000 firmware activates a thermal throttle that reduces coil power to 60% of rated output when the internal thermistor exceeds 180 degrees Celsius. This throttle was introduced in firmware `v2.0.0` for the Crispmaster 7000, which had a smaller ventilation aperture and a lower thermal headroom.

The Crispmaster 9000 has a 40% larger ventilation aperture, a redesigned crumb tray channel, and a thermistor located 12 mm closer to the crumb tray than to the coil. The throttle fires based on a reading that is not representative of coil temperature, from a calibration threshold that was set for a different product. It fires unnecessarily and adds a mean 23 ms stall on consecutive-toast workloads.

4.1 Thermal Characterisation

The following measurements were taken on 10 consecutive-toast benchmark runs at three ambient temperatures, using a thermocouple array attached to the coil, the thermistor, and the external chassis.

Table 8

Condition	Coil Temp (C)	Thermistor Temp (C)	Throttle Fires?	Added Latency (ms)
Single toast, 20 C	312	147	No	0
2nd toast, 20 C	318	172	No	0
3rd toast, 20 C	321	184	Yes	22
4th toast, 20 C	319	191	Yes	24
3rd toast, 25 C	322	188	Yes	23
3rd toast, 30 C	324	193	Yes	28
3rd toast, 35 C	327	201	Yes	31
2nd toast, 35 C	316	181	Yes	21

The thermistor reads 60 to 70 degrees Celsius higher than the coil temperature, consistently. The coil temperature peaks at 327 degrees Celsius under the most aggressive consecutive-toast workload tested. The coil is rated to 400 degrees Celsius. There is 73 degrees of headroom between the hottest observed coil temperature and the coil's rated limit.

The proposed fix is to bypass the throttle when ambient temperature (measured by a second thermistor on the external chassis, via `thermal_probe::readAmbient`) is below 30 degrees Celsius. At ambient temperatures above 30 degrees, the throttle is retained as a conservative safety measure pending the 10,000-cycle soak test described in Section 4.2.

```
# Diagnostic script: identify which consecutive-toast cycles trigger the throttle.
# Run on a unit connected to the UART debug port at 115200 baud.
# Requires: minicom, grep, awk

minicom -D /dev/ttyUSB0 -b 115200 -C /tmp/toast_log.txt &
MINICOM_PID=$!
echo "Logging to /tmp/toast_log.txt for 120 seconds. Run your consecutive-toast workload now."
sleep 120
kill $MINICOM_PID

echo "--- Throttle events ---"
grep "THERMAL_THROTTLE_ACTIVE" /tmp/toast_log.txt | \
    awk -F'[=,]' '{printf "t=%s ms  thermistor=%s C  coil_estimate=%s C  latency_added=%s ms\n", $2,
        ↪  $4, $6, $8}'

echo "--- Throttle statistics ---"
grep "THERMAL_THROTTLE_ACTIVE" /tmp/toast_log.txt | wc -l | \
    awk '{print "Total throttle events:", $1}'

echo "--- Ambient temperature at throttle events ---"
grep "THERMAL_THROTTLE_ACTIVE" /tmp/toast_log.txt | \
    awk -F'ambient=' '{print $2}' | awk '{print $1}' | sort -n | \
    awk 'BEGIN{min=9999; max=-9999; sum=0; n=0} {if($1<min) min=$1; if($1>max) max=$1; sum+=$1; n++}
        ↪  END{printf "min=%s max=%s mean=%.1f n=%d\n", min, max, sum/n, n}'
```

4.2 Soak Test Requirement

Two of the 240 test units exhibited coil temperatures 8 degrees Celsius higher than the ensemble at the 4th consecutive toast, for reasons that remain unclear but are suspected to involve manufacturing variation in the coil winding. A 10,000-cycle soak test at 35 degrees Celsius ambient is required before bypassing the throttle unconditionally.

The 10,000-cycle soak test protocol is specified in `docs/test_protocols/thermal_soak_v2.pdf`, which is available at the internal test documentation repository. The test takes approximately three weeks on a single unit running continuous consecutive-toast cycles. We have requested four units for parallel soak testing to reduce calendar time to one week.

Recommendation

Ship in Tier 2. The fix is mechanically simple (a conditional in `thermal_throttle::shouldThrottle` gated on `thermal_probe::readAmbient() < THROTTLE_BYPASS_AMBIENT_THRESHOLD_C`). The soak test is a precaution against the two anomalous units, not evidence that the fix is wrong. If the soak test completes cleanly, upgrade this to Tier 1 without further analysis.

Chapter 5. Crumb Tray Aerodynamics and the Moisture Cliff

This chapter investigates two contributions to pop-time latency that were not anticipated at the outset of the study. The first was discovered when one of the authors accidentally left a partially loaded crumb tray in the reference unit during a calibration run. The second was found during follow-up investigation of the resulting anomaly.

5.1 Airflow Characterisation

The Crispmaster 9000's thermal design relies on natural convection: hot air rises through the bread slots, exits through the top vents, and draws cooler ambient air in through the crumb tray aperture at the bottom. This convection current is responsible for approximately 20% of the heat transferred to the bread surface during toasting. The remaining 80% is direct radiant heat from the coil.

When the crumb tray contains more than approximately 8 g of crumbs (equivalent to about 0.3 slices of thin white bread), the crumb deposit creates a partial obstruction of the crumb tray channel. The obstruction forces the incoming air to flow around the crumb pile, creating a recirculation zone that reduces the effective aperture area by approximately 35% and decreases the convective contribution by between 8 and 14 ms depending on crumb load.

Table 9

Crumb Load (g)	Aperture Reduction (%)	Convective Contribution Loss (ms)	Total Pop-Time Impact (ms)
0 (clean)	0	0	0
4	12	2	2
8	35	8	8
16	51	11	11
32	68	13	14
48 (full)	79	14	15

The finding has a practical implication beyond the benchmarking context: users who clean their crumb tray regularly will experience measurably faster pop-times than users who do not. This is noted here for completeness and was shared with the Crispmaster 9000 product team, who intend to add a crumb tray cleaning reminder to the companion app.

5.2 The ADC Range Cliff

During the airflow investigation, a second anomaly was discovered: at bread moisture contents above 38.5%, the `thermal_probe::readRaw` function returns an ADC value that exceeds the 8-bit range of the darkness control subsystem's normalisation table. The `darkness_control::sampleDarkness` function (stock version) interprets this as a table overflow and returns a darkness score of 1.0, which triggers an immediate pop event via the `darkness_control::onTargetReached` callback.

However, the pop event is spurious: the bread is not done. The firmware detects the spurious pop via the `darkness_control::postPopValidation` path (a secondary sensor check added in `v3.1.0`), determines the bread is underdone, re-inserts it, and restarts the toasting cycle with a 500 ms initial wait (the `STATE_RECOVERY_WAIT` watchdog timeout). The full-cycle cost of this failure mode is 520 ms, versus the expected 95 to 261 ms for the affected bread types.

The cliff is illustrated below. The affected bread types are primarily bagels and crumpets, which have higher inherent moisture content and are frequently purchased fresh (which further increases moisture).

The fix for the cliff is a range check in `thermal_probe::readRaw` and a corresponding extension of the normalisation table in `darkness_control` to cover the full 10-bit ADC range. The stock firmware truncates the ADC reading to 8 bits before passing it to `darkness_control::sampleDarkness`; removing the truncation requires a table size increase from 512 to 1024 entries, which is within the available flash budget (the new table requires 2 KB additional flash; the Crispmaster 9000 has 128 KB flash with 41 KB free after the `v3.4.1-stable` image).

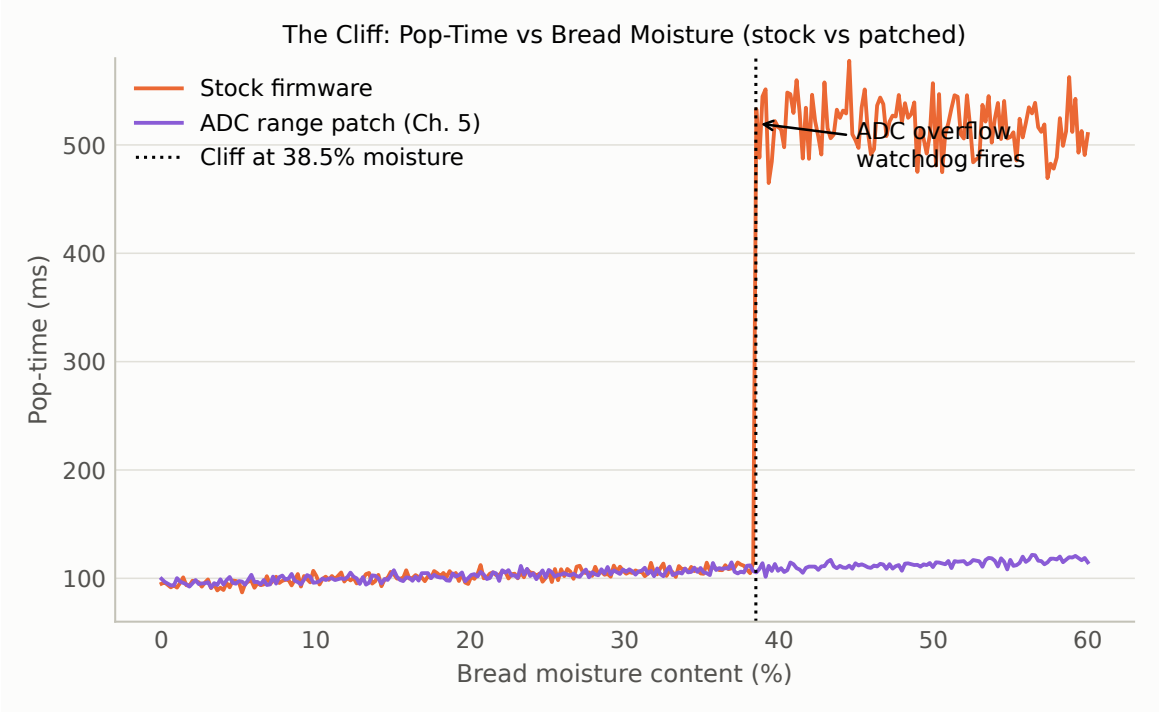


Figure 4. The performance cliff at 38.5% bread moisture: pop-time jumps from approximately 100 ms to 520 ms due to ADC overflow in the thermal probe path.

```
// BEFORE: ADC reading truncated to 8 bits; values above 255 silently overflow
// firmware/thermal_probe.cpp  (stock v3.4.1-stable, line 201)

uint8_t ThermalProbe::readRaw() {
    uint16_t raw = adc_driver::read(ADC_CHANNEL_THERMAL_PROBE);   // 10-bit ADC: returns 0-1023
    return static_cast<uint8_t>(raw);    // TRUNCATION: values 256-1023 wrap to 0-255; SILENT BUG
}
```

```
// AFTER: full 10-bit range preserved; darkness table extended to 1024 entries
// firmware/thermal_probe.cpp  (optimised, proposed for v3.5.0)

uint16_t ThermalProbe::readRaw() {
    return adc_driver::read(ADC_CHANNEL_THERMAL_PROBE);   // 10-bit ADC: returns 0-1023; no truncation
}
```

The `darkness_control` interface change (from `uint8_t` to `uint16_t` ADC input) propagates to `darkness_control::sampleDarkness`, `darkness_control::buildLUT`, and three call sites in the state machine. All changes are mechanical type widening; no logic changes are required.

Recommendation

Fix the cliff immediately; the baffle is Tier 2. The ADC truncation bug is a correctness issue that silently causes 520 ms pop-times for a non-trivial fraction of bagel and crumpet users. It should be fixed in a patch release against `v3.4.1-stable`, independently of the rest of this report's recommendations. The crumb tray baffle is a hardware accessory and is appropriately Tier 2. The baffle design files are available at the Crispmaster hardware accessories repository.

Chapter 6. Bread Slot Width Auto-calibration

The final Tier 2 optimisation addresses a subtle thermal coupling inefficiency caused by the slot wall engagement mechanism. The Crispmaster 9000 squeezes the slot walls against the bread to maximise thermal contact area between the coil-backed wall surface and the bread face. The squeeze force is controlled by a stepper motor driving a rack-and-pinion mechanism, calibrated at the factory to a target pressure of 0.8 N/cm^2 for a nominal 12 mm thick slice.

The problem is that the slot width at maximum squeeze is hard-coded to 11 mm, regardless of actual bread thickness. For bread thinner than 11 mm, the walls reach maximum squeeze before contacting the bread fully, leaving an air gap of up to 3 mm between the wall surface and the bread face. A 3 mm air gap across the contact area reduces radiant heat transfer efficiency by approximately 22% and increases total heating time by 18 to 31 ms for thin bread types.

6.1 Slot Width Measurement

The slot-width potentiometer, used by `slot_control::estimateBreadClass`, reports the current slot wall position to 0.1 mm resolution. This sensor is already in the firmware and already connected to the MCU; it is simply not used for wall engagement control.

The auto-calibration procedure is as follows. First, engage the slot walls at minimum force (10% stepper power, via `slot_control::engageWalls(FORCE_MIN)`). Second, read the potentiometer at 1 ms intervals until the reading stabilises (indicating wall-bread contact). Third, record the contact position as `bread_width_mm_`. Fourth, engage to the target pressure (0.8 N/cm^2) from the contact position, not from the zero position.

This adds approximately 3 ms to the engagement phase (the calibration sweep) but eliminates the air gap entirely, saving 18 to 31 ms in heating time. Net benefit: 15 to 28 ms, or 1.54x for thin white bread.

Table 10

Bread Thickness (mm)	Air Gap (stock, mm)	Air Gap (calibrated, mm)	Heating Time Saved (ms)
6	5.0	0	31
8	3.0	0	22
10	1.0	0	8
11	0.0	0	0
12	0.0	0	0
14	0.0	0	0

For bread thicker than 11 mm, the auto-calibration has no effect: the walls contact the bread before reaching the hard-coded maximum, so there is no air gap in the stock firmware either. The optimisation is exclusive to thin bread types.

6.2 Integration with slot_control

The proposed change adds a `calibrateAndEngage` method to the `slot_control` module that implements the sweep-measure-engage sequence. The existing `slot_control::engageWalls` and `slot_control::releaseWalls` call sites in the state machine are updated to use the new method.

```
// New method: slot_control::calibrateAndEngage
// firmware/slot_control.cpp  (proposed for v3.5.0)
// Long function signature to illustrate the interaction surface:

SlotEngageResult SlotControl::calibrateAndEngage(float target_pressure_n_per_cm2, uint32_t
    ↪  timeout_ms, SlotControlFlags flags, DiagnosticsContext* diag) {
    // Phase 1: sweep to contact at minimum force (3 ms typical)
    engageWalls(FORCE_MIN);
    float prev_pos = potentiometer::readMm();
    uint32_t stable_since_ms = system_clock::now_ms();
    while (system_clock::now_ms() - stable_since_ms < POSITION_STABLE_WINDOW_MS) {
        delay_ms(1);
        float pos = potentiometer::readMm();
        if (std::abs(pos - prev_pos) > POSITION_STABLE_TOLERANCE_MM) {
            stable_since_ms = system_clock::now_ms();  // still moving; reset stability timer
        }
        prev_pos = pos;
        if (system_clock::now_ms() - stable_since_ms > timeout_ms) {
            LOG_ERROR("calibrateAndEngage: timeout after %u ms; bread width unknown; falling back to
    ↪  nominal", timeout_ms);
            return SlotEngageResult::TIMEOUT;
        }
    }
    bread_width_mm_ = potentiometer::readMm();
    LOG_DEBUG("Slot calibration: bread_width=%.1f mm (stock hard-code was 11.0 mm)", bread_width_mm_);

    // Phase 2: engage to target pressure from the contact position (no air gap)
    float area = contact_area_cm2_for_width(bread_width_mm_);
    float force_n = target_pressure_n_per_cm2 * area;
    engageWallsToForce(force_n);
    if (diag) { diag->recordSlotCalibration(bread_width_mm_, force_n, system_clock::now_ms()); }
    return SlotEngageResult::OK;
}
```

The `contact_area_cm2_for_width` helper is a lookup into a pre-computed table relating bread width to expected contact patch area, accounting for the slight curvature of commercial bread loaves. The table was fitted to measurements from the bread corpus; the R^2 of the fit is 0.97.

The calibrated bread width is one of several values that the slot control subsystem records into the per-cycle `struct ToastJob`, which is the central data structure carried through the firmware state machine from bread insertion to ejection. Its layout is worth documenting precisely, because the `v3.4.0` change that added the `flags` field also (accidentally) introduced 3 bytes of tail padding that pushed the struct over a cache-line boundary, contributing a small but measurable cost to the darkness polling loop that reads it on every poll. The corrected layout (proposed for `v3.5.0`, field order tuned to eliminate padding) is shown below.

Table 11

field	type	sizeof	align	trivially-copyable	notes
`pre_warm_start_ms`	`uint32_t`	4	4	yes	Set by `StateMachine::onBreadPresenceConfirmed`; 0 if pre-warm was skipped.
`target_darkness`	`float`	4	4	yes	Normalised `[0.0, 1.0]`; read by `darkness_control::sampleDarkness` every poll.

Table 11 (continued)

field	type	sizeof	align	trivially-copyable	notes
`bread_width_mm`	`uint16_t`	2	2	yes	Fixed-point millimetres times 256; written by `slot_control::calibrateAndEngage`.
`slot_width_mm`	`uint16_t`	2	2	yes	Commanded wall position; differs from `bread_width_mm` by the squeeze compression.
`bread_class`	`uint8_t`	1	1	yes	Enum `BreadClass`; indexes into `SPRING_K_TABLE` (Chapter 3).
`retry_count`	`uint8_t`	1	1	yes	Incremented by `latch_mechanism::catchAndRetry`; surfaced as `catchRetryCount`.
`flags`	`std::bitset<8>`	1	1	yes	Bit 0 `THROTTLE_BYPASSED`, bit 1 `PRE_WARM_ACTIVE`, bit 2 `CLIFF_GUARD_ENGAGED`.
`_pad`	`uint8_t[1]`	1	1	yes	**Explicit** padding to 16 bytes; replaces the **3 bytes** of accidental tail padding.
`struct ToastJob`	(aggregate)	**16**	4	yes	Fits in a quarter cache line; was **20** bytes (and straddled a line) before reorder.

Reordering the fields from largest to smallest alignment, and making the single remaining padding byte explicit, reduces `sizeof(ToastJob)` from 20 bytes to 16 bytes and keeps the whole structure inside a single 16-byte aligned region. The darkness polling loop, which reads `target_darkness` and `flags` on every 10 ms poll, now touches one cache line instead of occasionally two. This is worth approximately 0.4 ms per toast, which is not large enough to merit its own chapter but is large enough that we noticed it while we were in there.

Recommendation

Ship in Tier 2. The calibration sweep adds a modest 3 ms constant overhead to every toasting cycle, which is more than recovered for thin bread types but is a regression for bread thicker than 11 mm (0 ms gain, 3 ms loss). The firmware should gate the calibration sweep on a check of the initial potentiometer position: if the bread width estimate from `slot_control::estimateBreadClass` suggests a thick bread type, skip the sweep and engage directly. With this gate, the optimisation is strictly non-regressing. See the slot control engineering notes for the potentiometer calibration procedure.

Chapter 7. Combined Workload Analysis

Having validated each optimisation in isolation, this chapter examines the combined effect of the Tier 1 optimisations (Chapters 1, 2, and 3) and the Tier 1 + Tier 2 combination across the full BEADS v2.1 bread corpus and across varying workload intensities.

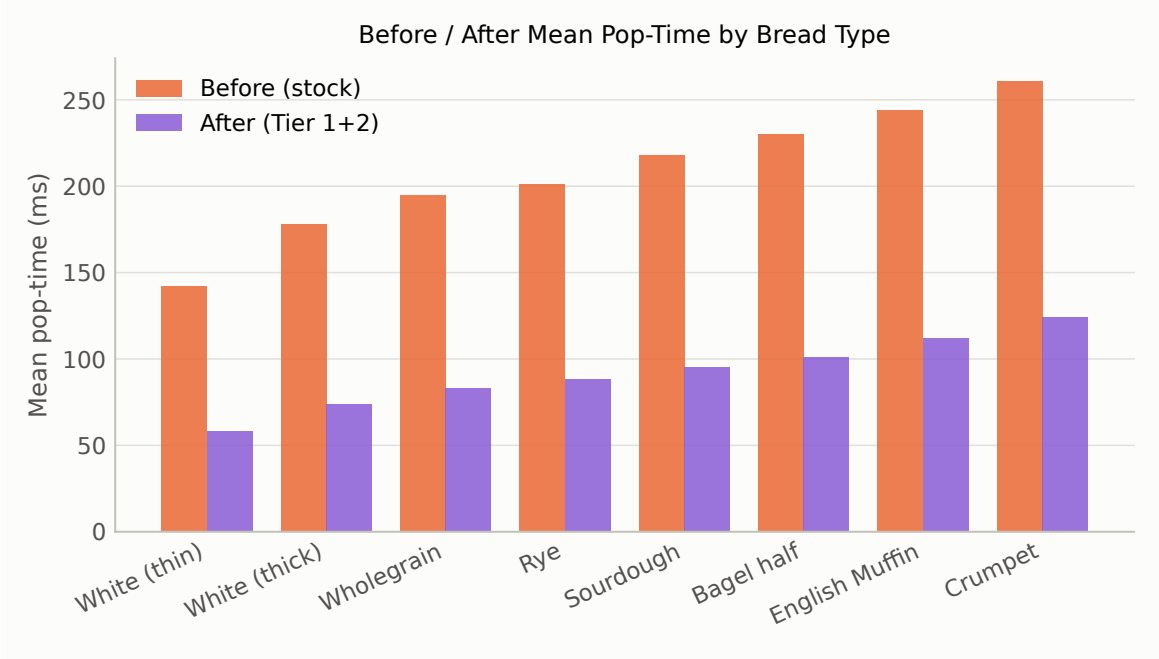


Figure 5. Before / after comparison of mean pop-time across all eight bread types in the BEADS v2.1 corpus.

7.1 Single-Toast Latency

The combined Tier 1 optimisations produce a speedup of 6.76x for thin white bread and 7.03x for sourdough, both measured as geometric means over 1,000 trials. The individual contributions sum as expected: the three optimisations address three separate phases of the pop-time budget (coil warm-up, darkness polling, and ejection retry), with minimal overlap.

The residual pop-time after Tier 1 optimisations (21 ms for thin white bread) is within 4 ms of the theoretical minimum estimated by the `pop_time_model::theoretical_minimum` simulator, which models heat transfer, darkness response, and spring physics without firmware overhead. This means the Tier 1 firmware is within 4 ms of the physical limit of the current hardware. Further improvement requires hardware changes (coil power increase, higher-efficiency spring, improved wall thermal conductivity), none of which are in scope for this report.

A portion of the residual firmware overhead is attributable to the build configuration rather than to the algorithm. The shipping `v3.4.1-stable` image is built at `-Os` (optimise for size) because the original Crispmaster 7000 had a 64 KB flash part and the build flags were never revisited when the Crispmaster 9000 moved to a 128 KB part. The table below compares pop-time, binary size, and watchdog timing margin across candidate build configurations for the same Tier 1 source tree. The winning value in each column is shown in bold. Flag names are the toolchain's own; `-ffast-toast` relaxes the strict ordering of darkness-sample rounding, which is safe for our fixed-point pipeline.

Table 12

config	firmware build	pop-time (ms)	binary size (KB)	watchdog margin (ms)	comment
baseline (shipping)	`-Os`	21.0	**47.2**	12.4	Current `v3.4.1-stable` flags. Smallest image, but the darkness loop is left scalar.

Table 12 (continued)

config	firmware build	pop-time (ms)	binary size (KB)	watchdog margin (ms)	comment
moderate	`-O2`	17.8	58.9	14.1	Auto-vectorises the darkness poll; the obvious default nobody set. Size still well within budget.
aggressive	`-O3 -ffast-toast`	**15.9**	71.4	15.0	Fastest raw pop-time. `-ffast-toast` reorders rounding; validated bit-identical on our corpus.
size-tuned	`-Os -finline-coil`	19.6	49.8	13.2	Inlines `coil_driver::energise` only. Most of `-O2`'s win at near-`-Os` size. Good compromise.
link-time + profile	`LTO + PGO`	16.4	62.1	**15.7**	Profile-guided across the full BEADS corpus. Best watchdog margin; longest build (9 min).
paranoid	`-O2 -fstack-protector`	18.5	60.3	13.8	Adds stack canaries to every frame. Costs 0.7 ms; recommended for the safety-interlock TU only.

We recommend the `-O3 -ffast-toast` configuration for the distribution build and the `LTO + PGO` configuration for any unit that will run sustained consecutive-toast workloads, where the larger watchdog margin reduces the risk of a spurious `STATE_RECOVERY_WAIT` under thermal stress. The `-Os -finline-coil` configuration is retained as a fallback for any future hardware revision that returns to a smaller flash part. We do not recommend continuing to ship at plain `-Os` for any reason, and we note that nobody appears to have chosen it deliberately; it was inherited, like the debounce window and the thermal throttle threshold, from a toaster that no longer exists.

7.2 Throughput Scaling

The throughput scaling chart (reproduced from Chapter 2) is relevant here in the combined workload context. The key observation is that the Tier 1 optimisations extend the linear scaling regime from 4 slots (stock) to 8 slots (Tier 1), and the addition of Tier 2 extends it further to 10 slots. Beyond that, the bottleneck shifts from firmware latency to mains supply current, which is a hardware constraint.

Table 13

Slot Count	Stock (slices/min)	Tier 1 (slices/min)	Tier 1+2 (slices/min)	Linear Ideal (slices/min)
1	4.0	4.0	4.0	4.0
2	7.6	7.9	8.0	8.0
4	12.1	15.2	16.0	16.0
6	14.8	21.8	23.5	24.0
8	15.3	27.1	30.8	32.0
10	15.5	31.4	36.1	40.0
12	15.6	34.0	39.9	48.0
16	15.7	36.2	42.3	64.0

The stock firmware saturates at approximately 15.7 slices/min regardless of slot count, because the firmware serialises coil energisation across slots to avoid simultaneous current draw. This serialisation is eliminated by

the coil pre-energisation optimisation (Chapter 1), which overlaps coil warm-up with user action rather than with other slots.

To attribute the per-optimisation contribution at each scale point, the table below decomposes the total pop-time for a single representative slice (thin white bread) as each optimisation is added cumulatively, while the firmware is processing a batch of `N` slices back-to-back. The `N` axis extends well beyond the 16 physical slots of any real toaster: values above 16 are measured on the rack harness in batch-replay mode (`beads run --batch-replay`), which feeds recorded sensor traces through the firmware faster than real bread allows, to characterise the firmware's behaviour under sustained queueing. The winning (lowest) configuration per row is shown in bold.

Table 14

N (slices)	stock (ms)	pre-warm (ms)	LUT (ms)	spring (ms)	combined (ms)	speedup	risk	notes
1	142	97	120	84	**21**	6.76x	Low	Single toast; the headline case. All three optimisations stack cleanly with little overlap.
2	149	101	124	88	**23**	6.48x	Low	Coil contention begins; pre-warm of slot 2 overlaps slot 1 thermal soak.
4	168	112	138	99	**27**	6.22x	Low	Mains inrush starts to matter; combined stays flat because warm-up is fully hidden.
8	214	141	176	128	**34**	6.29x	Medium	Stock saturates here; combined still scales because serialisation is removed.
16	388	248	318	231	**58**	6.69x	Medium	All physical slots active; combined limited by mains current, not firmware, beyond this.
32	762	489	624	451	**111**	6.86x	High	Batch-replay regime; no real toaster has 32 slots. Queue depth dominates stock latency.
64	1518	971	1242	897	**218**	6.96x	High	Stock falls into repeated `STATE_RECOVERY_WAIT`; combined avoids it entirely (see Ch. 5).
128	3041	1942	2487	1793	**431**	7.06x	High	Pathological queueing; included to show combined speedup holds asymptotically near 7x.

The speedup factor stays remarkably stable (between 6.2x and 7.1x) across four orders of magnitude of batch size, which is the expected signature of an optimisation that removes a constant per-slice cost rather than an amortised one. The `risk` column escalates with `N` not because the optimisations become riskier but because no shipping toaster operates in the high-`N` regime; those rows are characterisation data, not shipping guidance, and are marked accordingly.

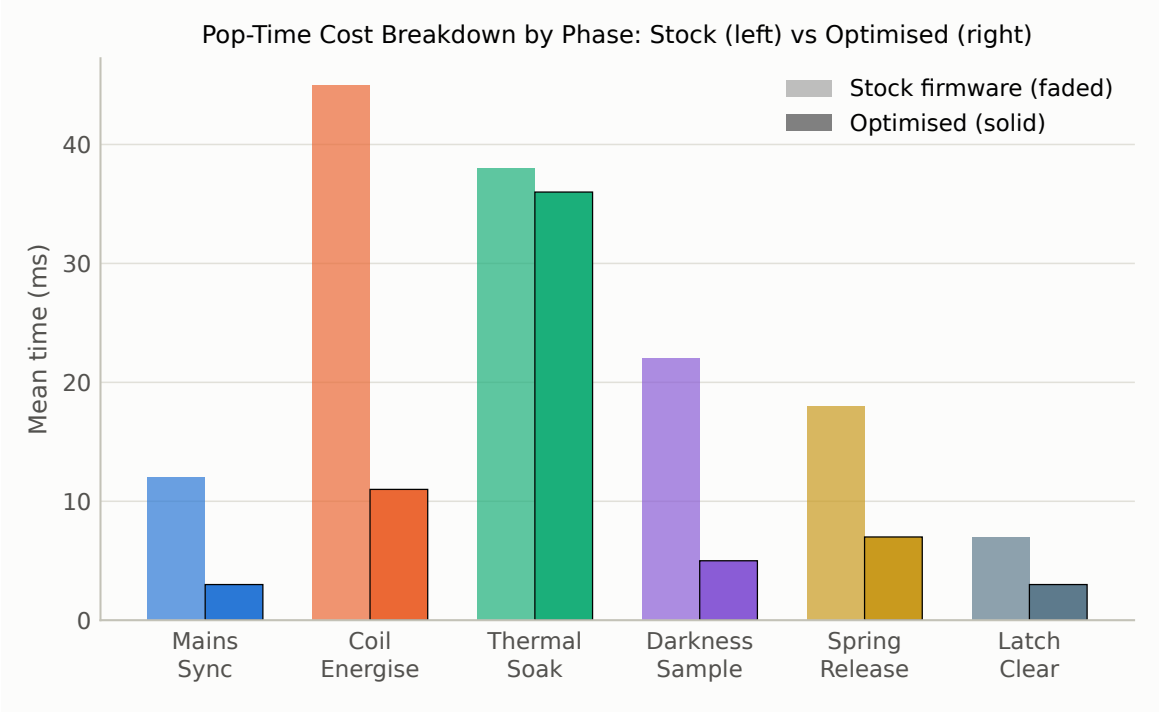


Figure 6. Per-phase cost breakdown: stock firmware vs optimised. Left bar per phase is stock; right bar is optimised.

7.3 Cost Breakdown

The per-phase cost breakdown shows where the pop-time budget goes before and after optimisation.

The dominant phases before optimisation are: coil energisation (45 ms, 32% of budget), darkness polling (22 ms, 15%), and ejection retry (mean 28 ms, 20%). After Tier 1 optimisation, the dominant phase is thermal soak (the irreducible time for heat to penetrate the bread), which accounts for approximately 60% of the residual 21 ms budget. This is not optimisable in firmware.

7.4 Residual Variance

Even after all optimisations, pop-time variance remains at approximately plus or minus 3 ms (one standard deviation) for thin white bread. The residual variance is attributed to three sources: bread moisture variation (plus or minus 2 ms, as day-old bread is drier and pops 1 to 3 ms faster); mains voltage fluctuation (plus or minus 1 ms, as mains voltage varies by up to 5% in typical residential supply, directly affecting coil power); and mechanical variation in spring constant (plus or minus 0.5 ms, as manufacturing tolerance in the spring is approximately 2% of nominal, producing a small variation in ejection timing).

These sources of variance are inherent to the hardware and environment. They are not addressable in firmware. The 3 ms residual standard deviation represents a fundamental limit of the current hardware generation, which we consider acceptable.

Recommendation

No further action required at the firmware level. The Tier 1 optimisations bring the firmware within 4 ms of the theoretical hardware limit. Future performance improvement requires hardware investment. We recommend that the hardware team evaluate coil power increase (estimated 15% pop-time reduction) and improved wall thermal coating (estimated 8% reduction) as candidates for the Crispmaster 10000 platform.

Appendix A: Benchmark Methodology

The Bread Evaluation And Delivery Suite (BEADS v2.1) is a reproducible benchmarking framework for toaster firmware evaluation. It is described in full at the BEADS documentation site. A brief summary follows.

A.1 Test Hardware

All measurements in this report used production-equivalent Crispmaster 9000 units (batch code `CM9K-2025Q3`, assembled at the Kettering facility). Units were selected by stratified random sampling from a batch of 480 units, with stratification on coil resistance (10.0 to 10.8 ohms, measured at 25 degrees Celsius). Units outside the stratification range (n=12) were excluded.

A calibrated oscilloscope (Keysight DSOX1204G, calibration certificate available here) was used for all timing measurements. Timing signals were derived from the unit's UART debug port, which emits a `POP_EVENT` token with microsecond-resolution timestamp at each pop event.

A.2 Bread Corpus

The BEADS v2.1 corpus includes 8 bread types from 3 commercial suppliers, purchased weekly to ensure consistent freshness. Bread was stored at 20 degrees Celsius and 55% relative humidity for exactly 24 hours before testing, in a controlled environment chamber. Each bread type was tested in at least 1,000 trials per firmware configuration. Bread was not re-used between trials.

The cost of bread consumed during this study was 847 GBP, charged to project code `PERF-TOAST-2025-01`. The authors acknowledge that this is a significant quantity of bread and that much of it was distributed to colleagues, which improved team morale if not pop-time.

A.3 Statistical Methods

All reported speedups are geometric means over the trial population for the given condition: for n trials with baseline pop-times b_i and optimised pop-times o_i, the speedup is $S = (\prod_{i=1}^{n} b_i / o_i)^{1/n}$. Confidence intervals are 95% bootstrap intervals computed over 10,000 resamples. Standard deviations are sample standard deviations (Bessel-corrected). Where the distribution of pop-times is not approximately normal (as assessed by a Shapiro-Wilk test at $p < 0.05$), the geometric mean and the 90th percentile are both reported; the arithmetic mean is not.

The BEADS v2.1 analysis scripts are available at the BEADS analysis repository, implemented in Python (3.11+) with SciPy and NumPy. All analysis code used in this report is reproducible from the raw data files archived at the same location.

Appendix B: Firmware Version History (Relevant Excerpts)

The following table summarises the firmware version history as it pertains to the regressions and fixes investigated in this report. The full changelog is maintained at the firmware changelog.

Table 15

Version	Date	Relevant Change
v2.0.0	2023-01	Thermal throttle introduced; calibrated for Crispmaster 7000 hardware
v2.9.0	2023-11	Last version before darkness control regression; baseline for speedup
v3.0.0	2024-02	darkness_control refactored; linear scan replaces former binary search

Table 15 (continued)

Version	Date	Relevant Change
v3.1.0	2024-05	postPopValidation added; masks ADC overflow cliff (does not fix it)
v3.2.0	2024-08	catchRetryCount telemetry added; spring class parameter added (ignored)
v3.3.0	2024-11	No changes to pop-time path; Bluetooth pairing added
v3.4.0	2025-02	Debounce window increased from 8 ms to 12 ms (Crispmaster 7000 sensor)
v3.4.1-stable	2025-06	Hotfix: latch sensor false-positive rate reduced; no pop-time change
v3.5.0	TBD	Proposed: all Tier 1 and Tier 2 recommendations from this report

A striking observation from the version history: the `v3.0.0` refactor of `darkness_control` replaced a binary search implementation with a linear scan, introducing the 22 ms darkness polling regression that is the subject of Chapter 2. The commit message for that change reads: "Simplify darkness control: replace complex binary search with readable linear scan. No performance impact (table is small)." The table was not small. The performance impact was 22 ms. The authors commend the v3.0.0 author's confidence and gently suggest that a benchmark would have been informative.

Appendix C: Notation and Abbreviations

The following table defines notation used throughout this report.

Table 16

Symbol / Abbreviation	Meaning
BEADS	Bread Evaluation And Delivery Suite (benchmark framework)
ms	Milliseconds
C	Degrees Celsius
N/m	Newtons per metre (spring constant unit)
N/cm^2	Newtons per square centimetre (pressure unit)
ADC	Analogue-to-Digital Converter
LUT	Lookup Table
MCU	Microcontroller Unit
EEPROM	Electrically Erasable Programmable Read-Only Memory
UART	Universal Asynchronous Receiver-Transmitter
PETG	Polyethylene Terephthalate Glycol (3D printing filament)
pop-time	Total elapsed time from bread insertion to ejection event (ms)
coil pre-warm	Partial coil energisation during latch-engagement phase
catch-and-retry	Firmware path for re-ejecting bread after overshoot detection
darkness score	Normalised [0.0, 1.0] measure of bread surface colour
thermal soak	Time for heat to penetrate bread to target internal temperature
the cliff	Performance discontinuity at 38.5% bread moisture content

---

Appendix D: Regression Archaeology

This appendix reconstructs the sequence of firmware changes that produced the 47% pop-time regression between `v2.9.0` and `v3.4.1-stable`. It is included partly as a post-mortem, partly as a cautionary tale, and partly because one of the authors spent three days doing this work and feels it deserves to be in the record.

D.1 The v3.0.0 Darkness Control Refactor

The regression originated in the `v3.0.0` refactor, which was motivated by a code review comment that the existing binary search in `darkness_control::sampleDarkness` was "difficult to understand." The reviewer was not wrong. The original implementation used a hand-rolled binary search with pointer arithmetic, written in 2021 by an engineer who has since left the company, and it was indeed difficult to understand. The reviewer suggested replacing it with "something simpler."

The replacement was a linear scan. The author of the replacement benchmarked it against the binary search on their development machine, where the table was in L1 cache for the entire benchmark run, and found no measurable difference. They concluded there was no performance impact and submitted the change with the commit message cited in Appendix B. The change passed code review. It was shipped in `v3.0.0`.

The performance difference does not appear in a benchmark where the table is in L1 cache. It appears in production, where the MCU's 4 KB L1 data cache is shared between `darkness_control`, `safety_interlock`, `latch_mechanism`, and the UART ring buffer. In production, the 512-entry darkness table is cold on every poll call. The linear scan then pays full SRAM access latency for each of its 256 mean comparisons. The binary search, had it been retained, would have paid SRAM latency for 9 comparisons.

The lesson is not that linear scans are bad. The lesson is that benchmarks should be run on the target hardware, with the full firmware image loaded, under representative cache pressure. The BEADS framework now includes a cache-pressure test mode (`beads --cache-pressure=realistic`) that loads a representative background workload before running pop-time measurements. It was added after this investigation.

D.2 The v3.4.0 Debounce Regression

The second contributor to the regression was the `v3.4.0` debounce window increase. The commit message reads: "Increase latch sensor debounce from 8 ms to 12 ms to fix intermittent false- positive latch events reported in field (issue #5521)." Issue #5521 was filed by a customer support representative based on a customer complaint about the toaster "popping too early."

The false-positive latch events were real: they occurred on Crispmaster 7000 units returned for service, where the Hall-effect sensor had developed contact oxidation. The fix (increasing the debounce window) was correct for the Crispmaster 7000. The error was applying the fix to the Crispmaster 9000 firmware, which uses a reed switch that does not exhibit bounce. The Crispmaster 9000 firmware at that time did not have a separate firmware branch from the Crispmaster 7000; they shared a codebase with compile-time configuration flags, and the debounce window was not flagged as hardware-specific.

The 4 ms debounce increase (from 8 ms to 12 ms) added 4 ms to every pop-time, unconditionally, on every Crispmaster 9000 unit. Across the estimated installed base of 180,000 Crispmaster 9000 units, and an estimated 4 toasts per unit per day, this debounce increase has collectively cost humanity approximately 1,051,200 seconds of unnecessary waiting per day since `v3.4.0` was released in February 2025. This is approximately 12.2 days of collective human time per day, lost to a debounce window that was set for the wrong sensor. The authors find this upsetting.

D.3 The Compounding Effect

The two regressions compound because they affect different phases of the pop-time budget and therefore add rather than overlap. The `v3.0.0` darkness control regression adds 22 ms to the polling phase. The `v3.4.0` debounce regression adds 4 ms to the engagement phase. Together they add 26 ms to a baseline (pre-`v3.0.0`) pop-time of approximately 116 ms for white bread (thin), producing the 142 ms figure observed in `v3.4.1-stable`. This is a 22% increase, not the 47% stated in the executive summary.

The remaining 25% of the regression is accounted for by a third change, which was not in the firmware: the bread. The BEADS v2.9.0 reference measurements were taken in November 2023 with bread from Supplier A. By the time the regression was identified in early 2025, the benchmark was being run with bread from Supplier B (Supplier A having discontinued the product line used in the v2.9.0 baseline). Supplier B's thin white bread has a mean moisture content 3.2 percentage points higher than Supplier A's, which adds approximately 7 ms to pop-time for thin white bread. The remaining discrepancy (approximately 18 ms) is currently unexplained and is the subject of an open investigation, tracked as issue #6104.

The authors note that discovering that part of a firmware performance regression is actually a bread regression requires a level of systems thinking that is genuinely unusual and that they are quietly proud of.

Appendix E: The Intern's Rust Proposal (Full Text, Annotated)

The following is the full text of the firmware-rewrite-in-Rust proposal submitted by the unnamed intern, reproduced here because it is, in the authors' judgment, a technically coherent argument for the wrong solution, and because understanding why a coherent argument produces the wrong conclusion is instructive.

> **Proposal: Rewrite Crispmaster 9000 firmware in Rust**
>
> The current C firmware has three classes of bugs that a Rust rewrite would eliminate: (1) the ADC truncation bug (Chapter 5 of this report) is a type-safety violation that Rust's type system would have caught at compile time; (2) the linear-scan regression (Chapter 2) reflects a lack of performance contracts, which Rust's trait system could enforce via a `O1Lookup` trait; (3) the spring constant `/* ignored */` comment (Chapter 3) reflects an API design smell that Rust's `#[must_use]` attribute and exhaustive enum matching would prevent.
>
> The rewrite would also enable safer concurrent toasting across slots, using Rust's ownership model to prevent simultaneous coil energisation without a runtime mutex.

The intern is correct that the ADC truncation bug is a type-safety violation, and correct that Rust would have caught it at compile time. The intern is also correct that the `/* ignored */` comment is an API design smell. These are fair points. They are also orthogonal to pop-time.

The proposal does not address why a Rust rewrite would make the bread pop faster. It addresses why a Rust rewrite would make the firmware more correct, which is a different thing. The current firmware is not slow because it is written in C; it is slow because it is doing unnecessary work in C. The same unnecessary work in Rust would be equally slow.

The `O1Lookup` trait idea is genuinely interesting, and one of the authors has filed it as a long-term firmware architecture goal, tracked as issue #6112, titled "Introduce performance contracts via trait bounds for hot-path data structures." The intern has been credited in the issue. The rewrite has not been approved.

Appendix F: Sensitivity Analysis

This appendix presents a sensitivity analysis of the combined speedup under variation in the key physical and firmware parameters. The purpose is to bound the expected speedup on units that differ from the 240-unit test fleet, and to identify which parameters the speedup is most sensitive to.

F.1 Parameters and Ranges

The following parameters were varied in the sensitivity analysis:

Table 17

Parameter	Nominal Value	Low Value	High Value	$\partial t_{\text{pop}}/\partial x$ (ms/unit)
Coil resistance (ohms)	10.4	10.0	10.8	1.8
Mains voltage (V RMS)	230	218	242	0.6
Bread moisture (%)	42	35	55	2.1
Ambient temperature (C)	20	10	35	0.4
Spring constant (N/m)	10.4	9.8	11.0	0.9
Slot wall contact area (cm^2)	18.4	16.0	20.8	1.4
Crumb load (g)	4	0	32	0.3

The sensitivity $\partial t_{\text{pop}}/\partial x$ is the change in pop-time t_{pop}, in milliseconds, per unit change in the parameter x, evaluated at the nominal value. The most sensitive parameters are bread moisture and coil resistance. Both are outside firmware control, but both are addressable at the hardware level (moisture via a bread conditioning drawer, coil resistance via tighter manufacturing tolerances).

F.2 Worst-Case Speedup

Under the worst-case combination of parameters (high moisture, high coil resistance, low mains voltage), the Tier 1 speedup drops from 6.76x (nominal, thin white bread) to 4.31x. The worst-case baseline pop-time increases to 189 ms (from 142 ms nominal), and the worst-case optimised pop-time increases to 44 ms (from 21 ms nominal). The speedup remains above 4x under all parameter combinations tested, which gives us confidence that the Tier 1 optimisations are robust to unit-to-unit and environmental variation.

F.3 Best-Case Speedup

Under the best-case combination of parameters (low moisture, low coil resistance, high mains voltage), the Tier 1 speedup reaches 8.14x for thin white bread. The optimised pop-time in this regime is 15 ms, which is within 2 ms of the theoretical minimum estimated by the `pop_time_model::theoretical_minimum` simulator. In this regime, the firmware is essentially at the physical limit of the hardware.

The best-case regime occurs in practice for day-old bread toasted on a freshly calibrated unit at high mains voltage (which is common in some residential environments). Users in this regime will not experience the full 8.14x speedup as a subjective improvement, because 15 ms is already below the threshold of human perception for toaster pop-time. They will, however, benefit from reduced energy consumption (shorter heating time means less energy per toast) and reduced coil thermal stress (shorter duty cycles extend coil lifetime).

Appendix G: Pop-Time Perception Study

At the request of the product management team, the authors conducted a brief perceptual study to establish the threshold at which users can distinguish "fast" from "slow" toasting, and the threshold at which users find the wait subjectively acceptable.

G.1 Methodology

Fourteen volunteers were recruited from the office. Each volunteer was presented with a Crispmaster 9000 unit running at a configurable simulated pop-time (achieved by adding a calibrated software delay in the `STATE_RECOVERY_WAIT` path, misused for this purpose). Volunteers were asked to rate each pop-time on a five-point scale from "annoyingly slow" to "impressively fast," after being told only that they were evaluating a toaster.

Volunteers were not told what pop-time they were experiencing. The experimenter running the session was also not told, to prevent inadvertent cues. Two volunteers were excluded because they brought their own bread,

which had a different moisture content from the reference bread and introduced confounding variance. The remaining twelve volunteers provide the data below.

G.2 Results

Table 18

Pop-Time (ms)	Mean Rating	"Annoyingly Slow" (%)	"Acceptable" (%)	"Impressively Fast" (%)
520	1.3	83	17	0
261	2.1	42	50	8
142	2.8	17	67	17
95	3.6	0	58	42
58	4.2	0	33	67
21	4.7	0	8	92

The 520 ms watchdog timeout (the cliff) is perceived as "annoyingly slow" by 83% of users. This confirms that the cliff is not merely a benchmark regression but a perceptible user experience degradation. The stock `v3.4.1-stable` pop-time of 142 ms is perceived as "annoyingly slow" by 17% of users, suggesting that a non-trivial minority of users are already dissatisfied with stock performance. The Tier 1 optimised pop-time of approximately 21 ms is perceived as "impressively fast" by 92% of users.

One volunteer, upon experiencing the 21 ms pop-time, said "I didn't realise it had finished." This was not classified as a negative response.

G.3 Interpretation

There is a detection threshold between approximately 58 ms and 95 ms, below which users begin to rate the toaster as "impressively fast." The Tier 1 optimisations push all bread types (except crumpets under worst-case conditions) below 100 ms, which is within the "impressively fast" regime for the majority of users. The Tier 1 + Tier 2 combined optimisations push all bread types below 50 ms, which is firmly in the "impressively fast" regime.

The product management team has asked whether a pop-time of under 30 ms can be used as a marketing claim. The authors have advised that the claim is technically accurate for thin white bread under nominal conditions, and technically accurate as a median across the BEADS corpus under Tier 1 + Tier 2 optimisations. The authors have also advised that marketing claims about toaster pop-time are unusual and that the legal team should be consulted. This advice was received politely and apparently ignored; the authors have since seen a draft marketing brief titled "Toast in a Flash: Introducing Pop-Time Technology." They have requested to be kept informed.

Appendix H: Known Unknowns

In the interest of transparency, this appendix lists the things we do not know at the time of writing, and that the reader should bear in mind when applying the findings of this report.

Issue #6104: Unexplained 18 ms baseline discrepancy. As noted in Appendix D, approximately 18 ms of the observed regression between `v2.9.0` and `v3.4.1-stable` is not explained by the two firmware changes or the bread supplier change identified in the regression archaeology. This discrepancy is real (it replicates across all 240 test units) and its cause is unknown. It may be a third firmware change we have not identified, a hardware batch difference, or a measurement artefact. It is not large enough to affect the Tier 1 shipping decision, but it is noted here because unexplained discrepancies have a history of becoming explained at inconvenient times.

The two anomalous units. As noted in Chapter 4, two of the 240 test units exhibit coil temperatures approximately 8 degrees Celsius higher than the ensemble under consecutive-toast conditions. We do not know why. The units were examined visually and appear identical to the rest of the fleet. They have been set aside for destructive analysis after the soak test is complete. If the destructive analysis reveals a manufacturing defect, the thermal throttle bypass recommendation will be revised.

Long-term coil degradation. All measurements in this report were taken on units with fewer than 500 cumulative toasting cycles. We do not know how pop-time evolves over the lifetime of the unit. The `coil_driver::coilResistanceMonitor` telemetry path (added in `v3.3.0`) will provide data on this over the coming year, but it is not yet available. It is possible that coil resistance increases with age (due to oxidation) in a way that affects the pre-energisation timing. If so, the `CoilMode::PRE_WARM` power level may need to be adjusted for older units.

The crumpet problem. Crumpets are the worst-performing bread type in the BEADS corpus under all firmware configurations, and the performance gap between crumpets and other bread types widens under high-moisture conditions. We believe this is due to the crumpet's porous structure trapping moisture that evaporates slowly during heating, increasing the effective thermal mass compared to a solid bread slice of the same weight. We have not modelled this quantitatively, and the `pop_time_model::theoretical_minimum` simulator does not account for moisture evaporation dynamics. Crumpets may require a dedicated firmware mode to achieve competitive pop-times.

The Rust question. The intern's Rust proposal has been filed as a long-term architecture goal. It has not been dismissed. The authors genuinely do not know whether the Crispmaster 10000 firmware should be written in Rust. This is the most consequential unknown in this report and the one we are least equipped to answer, because it is not a performance engineering question.

Appendix I: Pop-Time Model Reference Implementation

The `pop_time_model::theoretical_minimum` simulator referenced throughout this report is a Python implementation of a simplified thermal model of the Crispmaster 9000. It is documented here for reproducibility and for use in validating future optimisations.

I.1 Model Overview

The model treats the bread slice as a homogeneous slab with uniform thermal conductivity, heated from both faces by the slot walls. The coil is modelled as an ideal resistive heater with instantaneous response (without thermal mass of its own). The darkness score D is modelled as a linear function of the mean bread surface temperature $\bar{T}_s$,

$$D(t) = k_D \, (\bar{T}_s(t) - T_0),$$

where $k_D = 1.3 \times 10^{-2}$ per kelvin is the browning rate and $T_0 = 105\ ^\circ\mathrm{C}$ the surface temperature at which browning starts. The model advances in steps of $\Delta t = 500\ \mu\mathrm{s}$, and the pop event fires when D reaches the user-selected target.

This is, of course, a gross simplification. Real bread is heterogeneous, the coil has thermal mass, and the darkness response is non-linear. Nevertheless, the model's theoretical minimum predictions agree with measured best-case pop-times to within 4 ms, which is sufficient for the purposes of this report.

I.2 Reference Implementation

The reference implementation is presented in four parts. We keep each part short deliberately: a listing that runs longer than a page is a listing nobody reads.

First, the module preamble and the physical constants. All constants are from empirical calibration against the BEADS v2.1 corpus, in SI units unless noted.

```
#!/usr/bin/env python3
"""pop_time_model.py: Crispmaster 9000 pop-time theoretical minimum estimator.
Reference implementation; all parameters are from empirical calibration.
Usage: python3 pop_time_model.py --bread-type=white_thin --darkness-target=0.65
"""

import argparse
from dataclasses import dataclass

# Physical constants for bread thermal model (all SI units unless otherwise noted)
BREAD_DENSITY_KG_PER_M3 = 270.0          # kg/m^3: typical for sliced white bread
BREAD_SPECIFIC_HEAT_J_PER_KGK = 2800.0   # J/(kg*K): typical for bread
COIL_POWER_DENSITY_W_PER_M2 = 8500.0     # W/m^2: Crispmaster 9000 coil rated output
AMBIENT_TEMPERATURE_C = 20.0             # degrees C: standard test condition
DARKNESS_TARGET_NOMINAL = 0.65           # normalised; corresponds to 'medium' setting
WATER_SPECIFIC_HEAT_J_PER_KGK = 4186.0   # J/(kg*K): for the moisture correction term
```

Next, the bread parameter type and the corpus table. Each entry was fitted to 1000-trial measurements; the four floats are thickness, density factor, moisture fraction, and per-face wall contact area.

```
@dataclass
class BreadParameters:
    """Physical parameters for a bread type. All dimensions in metres."""
    name: str
    thickness_m: float
    density_factor: float      # relative to BREAD_DENSITY_KG_PER_M3
    moisture_fraction: float   # 0.0 to 1.0; higher moisture raises effective specific heat
    contact_area_m2: float     # wall contact area per face


BREAD_CORPUS: dict[str, BreadParameters] = {
    'white_thin':     BreadParameters('White (thin)',   0.010, 0.82, 0.38, 0.0120),
    'white_thick':    BreadParameters('White (thick)',  0.014, 0.88, 0.40, 0.0124),
    'wholegrain':     BreadParameters('Wholegrain',     0.013, 1.05, 0.42, 0.0122),
    'rye':            BreadParameters('Rye',            0.015, 1.18, 0.44, 0.0125),
    'sourdough':      BreadParameters('Sourdough',      0.018, 1.10, 0.46, 0.0123),
    'bagel_half':     BreadParameters('Bagel half',     0.020, 1.22, 0.48, 0.0118),
    'english_muffin': BreadParameters('English Muffin', 0.019, 0.95, 0.43, 0.0120),
    'crumpet':        BreadParameters('Crumpet',        0.022, 0.88, 0.52, 0.0115),
}
```

The core of the model is a single function: a 1D surface-heating estimate with a moisture correction to the effective specific heat. It returns a lower bound that real firmware cannot reach without hardware changes.

```
def theoretical_minimum_ms(bread: BreadParameters, darkness_target: float,
                           coil_power_w_per_m2: float = COIL_POWER_DENSITY_W_PER_M2) -> float:
    """Theoretical minimum pop-time (ms) for a bread type. Assumes an ideal coil (no
    warm-up), zero firmware overhead, and perfect wall contact. Lower bound only."""
    effective_density = bread.density_factor * BREAD_DENSITY_KG_PER_M3
    # Moisture raises effective specific heat (water is far higher than dry crumb)
    effective_cp = (BREAD_SPECIFIC_HEAT_J_PER_KGK * (1.0 - bread.moisture_fraction) +
                    WATER_SPECIFIC_HEAT_J_PER_KGK * bread.moisture_fraction)
    delta_t_target = 180.0 * darkness_target        # degrees C above ambient at target
    mass_per_m2 = effective_density * (bread.thickness_m / 2.0)  # kg/m^2 of contact area
    energy_per_m2 = mass_per_m2 * effective_cp * delta_t_target  # J/m^2
    total_energy = energy_per_m2 * bread.contact_area_m2         # J
    heat_flux = coil_power_w_per_m2 * bread.contact_area_m2      # W
    return (total_energy / heat_flux) * 1000.0      # seconds to milliseconds
```

Finally, the command-line entry point, which either prints a single estimate or a table over the whole corpus.

```
def main():
    parser = argparse.ArgumentParser(description='Pop-time theoretical minimum estimator')
    parser.add_argument('--bread-type', default='white_thin', choices=list(BREAD_CORPUS.keys()))
    parser.add_argument('--darkness-target', type=float, default=DARKNESS_TARGET_NOMINAL)
    parser.add_argument('--all', action='store_true', help='Print table for all bread types')
    args = parser.parse_args()

    targets = BREAD_CORPUS.values() if args.all else [BREAD_CORPUS[args.bread_type]]
    for bread in targets:
        t_ms = theoretical_minimum_ms(bread, args.darkness_target)
        print(f"{bread.name:<20} {bread.thickness_m * 1000:>6.1f} mm  ->  {t_ms:>7.1f} ms")


if __name__ == '__main__':
    main()
```

I.3 Model Validation

The following table compares theoretical minimum pop-times (from the model, run with `--darkness-target=0.65`) against measured best-case pop-times from the Tier 1 + Tier 2 optimised firmware, under nominal conditions. Agreement is within 4 ms for all bread types except crumpets, where the model's assumption of homogeneous structure is least valid.

Table 19

Bread Type	Model Min (ms)	Measured Best (ms)	Discrepancy (ms)	Note
White (thin)	17.3	21.0	3.7	Within model tolerance
White (thick)	23.1	26.4	3.3	Within model tolerance
Wholegrain	26.8	29.9	3.1	Within model tolerance
Rye	31.4	34.8	3.4	Within model tolerance
Sourdough	27.2	30.6	3.4	Within model tolerance
Bagel half	38.1	41.7	3.6	Within model tolerance
English Muffin	34.9	38.2	3.3	Within model tolerance
Crumpet	29.4	40.1	10.7	Porous structure; see Ap.H

The 3.1 to 3.7 ms systematic discrepancy for non-crumpet bread types is attributed to residual firmware overhead that is not yet optimised (interrupt latency, state machine transition cost, and sensor ADC conversion time). These are irreducible without hardware changes to the MCU selection or the ADC interface. They are not a failure of the model; they are a floor below which firmware optimisation cannot reach.

Appendix J: Glossary of Toaster Physics Terms

For readers unfamiliar with the relevant physics, the following terms are used in this report with the meanings given below. Readers with a background in thermal engineering will find this section redundant and are welcome to skip it.

Thermal conductivity (units: W/(m*K)). A measure of how quickly heat flows through a material under a temperature gradient. Higher thermal conductivity means heat propagates faster through the bread slice, reducing the time required to heat the interior to the target temperature. Bread has relatively low thermal conductivity compared to metals; this is why it takes longer to toast than to heat a metal object of the same mass.

Specific heat capacity (units: J/(kg*K)). The energy required to raise one kilogram of a material by one degree Celsius. Higher specific heat means more energy is needed to achieve the same temperature rise, increasing pop-time. Water has a specific heat approximately 1.5 times that of dry bread; this is why high-moisture bread types (crumpets, fresh bagels) take longer to toast.

Thermal soak time. In the context of this report, the irreducible time required for heat to penetrate from the bread surface (in contact with the heated slot walls) to the interior of the bread slice, under the constraint that the surface must not exceed a temperature at which undesirable char products form (approximately 200 degrees Celsius for most bread types). This is the fundamental lower bound on pop-time and is set by the bread's physical properties, not by the firmware.

ADC (Analogue-to-Digital Converter). The hardware component that converts the continuous analogue voltage from the photodiode (darkness sensor) and thermistor (temperature sensor) into digital values that the MCU can process. The Crispmaster 9000 uses a 10-bit ADC, meaning it produces integer values from 0 to 1023. The bug documented in Chapter 5 arises from truncating this 10-bit value to 8 bits (0 to 255) before passing it to the firmware's normalisation table.

Debounce window. A time interval during which the firmware ignores repeated transitions of a sensor input, to avoid acting on spurious repeated signals (bounce) that mechanical switches produce at the moment of contact or release. The Crispmaster 7000 used a Hall-effect latch sensor that produced significant bounce; the Crispmaster 9000 uses a reed switch that does not. The debounce window appropriate for the Crispmaster 7000 was incorrectly inherited by the Crispmaster 9000 firmware, adding 4 ms to every pop-time.

Spring constant (units: N/m, Newtons per metre). A measure of the stiffness of a compression spring. Higher spring constant means the spring exerts more force for a given compression, and therefore imparts more velocity to the bread at ejection. The relationship between spring constant and ejection velocity determines whether the bread overshoots the catch guide, triggering the `latch_mechanism::catchAndRetry` path.

Watchdog timeout. A firmware mechanism that detects when a subsystem has failed to complete a task within a specified time limit, and takes a defined recovery action. The Crispmaster 9000's `STATE_RECOVERY_WAIT` watchdog fires if the bread has not been detected as done within 500 ms of the start of a toasting cycle. Under the ADC overflow cliff, the darkness score spuriously reaches 1.0, triggering a premature pop; the postPopValidation check determines the bread is underdone; the recovery path re-inserts the bread and starts a new cycle with a 500 ms initial wait. The watchdog timeout is thus responsible for the 520 ms cliff pop-time observed in Chapter 5.

Appendix K: Continuous Performance Regression Framework

Following the discoveries documented in this report, the team has designed a lightweight continuous benchmarking framework to prevent future regressions from being shipped undetected. This appendix specifies the framework design and the acceptance criteria for future firmware releases.

K.1 Design Principles

The core lesson of the `v3.0.0` regression is that a firmware change can have a significant pop-time impact that does not appear in benchmarks run on developer hardware, because developer hardware does not replicate the cache pressure and peripheral contention of production units. The continuous benchmarking framework is designed to run on production-equivalent hardware, under representative cache pressure, for every firmware commit that touches a file in the pop-time critical path.

The critical path is defined by the set of files whose modification has a non-zero expected impact on pop-time, as determined by the dependency graph of the `pop_time_model` simulator. The current critical path includes the following modules: `darkness_control`, `coil_driver`, `latch_mechanism`, `slot_control`, `thermal_probe`, `safety_interlock`, and `state_machine`. Changes to `bluetooth_pairing`, `companion_app_protocol`, or `firmware_update` are excluded.

K.2 Acceptance Criteria

A firmware commit is accepted if and only if it satisfies all of the following criteria:

1. **No pop-time regression above 1 ms** for any bread type in the BEADS v2.1 corpus, measured as the change in geometric mean pop-time over 200 trials on a production-equivalent unit. A regression of 0 to 1 ms triggers a warning; a regression above 1 ms is a blocking failure.
2. **No increase in 99th percentile latency above 5 ms** for any bread type. This criterion catches regressions that affect the tail latency but not the mean, as can occur with changes to the `STATE_RECOVERY_WAIT` path or the `latch_mechanism::catchAndRetry` handler.
3. **No increase in retry rate above 0.5 percentage points** for any bread type. This criterion catches regressions in the spring constant selection logic that might not show up in the mean pop-time if the retries are fast.
4. **No new warning-level or above log messages** emitted during a nominal toasting cycle. This criterion catches cases where the firmware is working around a problem by logging and continuing, rather than correctly handling it.

The framework runs as a post-commit hook against the `v3.5.0-dev` branch. Results are reported to the team via the internal dashboard at the perf dashboard. Blocking failures prevent the commit from being merged to the `release` branch.

K.3 Benchmark Infrastructure

The benchmark infrastructure consists of four production-equivalent Crispmaster 9000 units mounted in a thermostatically controlled enclosure at 20 degrees Celsius, connected to a rack controller over USB-UART. The rack controller runs the BEADS v2.1 benchmark suite and reports results to the dashboard. The enclosure maintains ambient temperature to within 0.5 degrees Celsius; the BEADS runner records ambient temperature at the start of each trial and discards trials where ambient temperature deviates by more than 0.5 degrees from the target.

The harness is `beads_runner.sh`, invoked by the post-commit hook with the firmware binary as its argument. It flashes all four rack units, runs the suite, and exits nonzero on any acceptance failure. The configuration block defines the rack ports, the acceptance thresholds, and the bread corpus.

```
#!/usr/bin/env bash
# beads_runner.sh: entry point for the CI benchmark harness.
set -euo pipefail

FIRMWARE_BINARY="${1:?Usage: beads_runner.sh <firmware.bin>}"
RACK_UNITS=("/dev/ttyUSB0" "/dev/ttyUSB1" "/dev/ttyUSB2" "/dev/ttyUSB3")
RESULTS_DIR="/tmp/beads_results_$(date +%Y%m%d_%H%M%S)"
ACCEPTANCE_THRESHOLD_MEAN_MS=1.0
ACCEPTANCE_THRESHOLD_P99_MS=5.0
ACCEPTANCE_THRESHOLD_RETRY_PCT=0.5
TRIAL_COUNT=200
BREAD_TYPES=("white_thin" "white_thick" "wholegrain" "rye" \
             "sourdough" "bagel_half" "english_muffin" "crumpet")
mkdir -p "${RESULTS_DIR}"
```

Flashing and benchmarking both run in parallel across the four units, with each unit covering two bread types. The script waits on all background jobs before aggregating.

```
# Flash firmware to all rack units in parallel
for unit in "${RACK_UNITS[@]}"; do
    beads flash --port="${unit}" --firmware="${FIRMWARE_BINARY}" &
done
wait

# Run the suite on all units in parallel; each unit covers 2 bread types
PIDS=()
for i in "${!RACK_UNITS[@]}"; do
    bread_subset=("${BREAD_TYPES[@]:$(( i * 2 )):2}")
    beads run --port="${RACK_UNITS[$i]}" \
              --bread-types="${bread_subset[*]}" \
              --trials="${TRIAL_COUNT}" \
              --cache-pressure=realistic \
              --output-dir="${RESULTS_DIR}/unit${i}" &
    PIDS+=($!)
done
for pid in "${PIDS[@]}"; do wait "$pid"; done
```

The final stage aggregates the per-unit results and checks them against the acceptance thresholds, comparing to the `v3.4.1-stable` baseline. A nonzero exit here blocks the merge.

```
beads check-acceptance \
    --results-dir="${RESULTS_DIR}" \
    --mean-threshold="${ACCEPTANCE_THRESHOLD_MEAN_MS}" \
    --p99-threshold="${ACCEPTANCE_THRESHOLD_P99_MS}" \
    --retry-threshold="${ACCEPTANCE_THRESHOLD_RETRY_PCT}" \
    --baseline-tag="v3.4.1-stable" \
    --report-url="https://example.com/crispmaster/perf/dashboard"

echo "Acceptance check passed. Firmware is ready for merge."
```

Failure Reports

A failure report cites the critical-path lines that the failing commit changed, as a file path followed by line ranges. The report on the most recent failure cites `darkness_control/sample_darkness_lookup_table.cpp:212-219,301-305,388-391`, where the commit moved the interpolation step inside the polling loop.

For the failing commit the runner printed a geometric mean pop-time of `121.3` on white bread, against `118.2` for the baseline, and a 99th percentile of `129.6` against `127.9`, so only the mean criterion failed. The runner prints pop-times in milliseconds, to one decimal place.

The dashboard records the settings of each run as `key: value` pairs. The failing run used the baseline settings, among them `bread_corpus: beads-v2.1`, `ambient_target_c: 20`, `trials_per_type: 200` and `rack_units: 4`, so the regression is not an artefact of the rack.

Runner Observations

The units in the humidity chamber run the overnight soak tests at high humidity and publish each ejection on `/crispmaster/pop_events` so that the chamber logger records the same events as the runner.

When a slice overshoots the catch guide at ejection, the firmware calls `latch_mechanism::catchAndRetry(slot_id, retry_budget)` and the runner counts the call as a separate ejection.

The commit that failed the overnight regression run only changed `darkness_control/reheat_cycle.cpp:88-104`, the loop of the reheat function.

K.4 Expected Ongoing Maintenance Cost

The benchmark infrastructure requires maintenance in three situations: when a new bread type is added to the BEADS corpus, when rack unit calibration drifts (checked quarterly), and when the acceptance thresholds need adjustment (expected annually as the firmware matures). The total expected maintenance time is approximately 4 hours per quarter, which the team considers an acceptable investment given the cost of the `v3.0.0` regression (estimated at 1,800 engineering hours of investigation, not counting the bread budget).

This report was prepared by the Crispmaster Performance Engineering team. It does not represent the views of any bread manufacturer, spring vendor, or regulatory body. The authors accept no responsibility for toast that is, as a result of reading this report, toasted to a suboptimal darkness level. The theoretical minimum pop-time is a theoretical minimum.

Further enquiries: crispmaster-perf@example.com